

Laporan M2 Tugas Besar 1

IF2010 Pemrograman Berorientasi Objek 2025/2026

Nimonspoli

Kode Kelompok : NWO

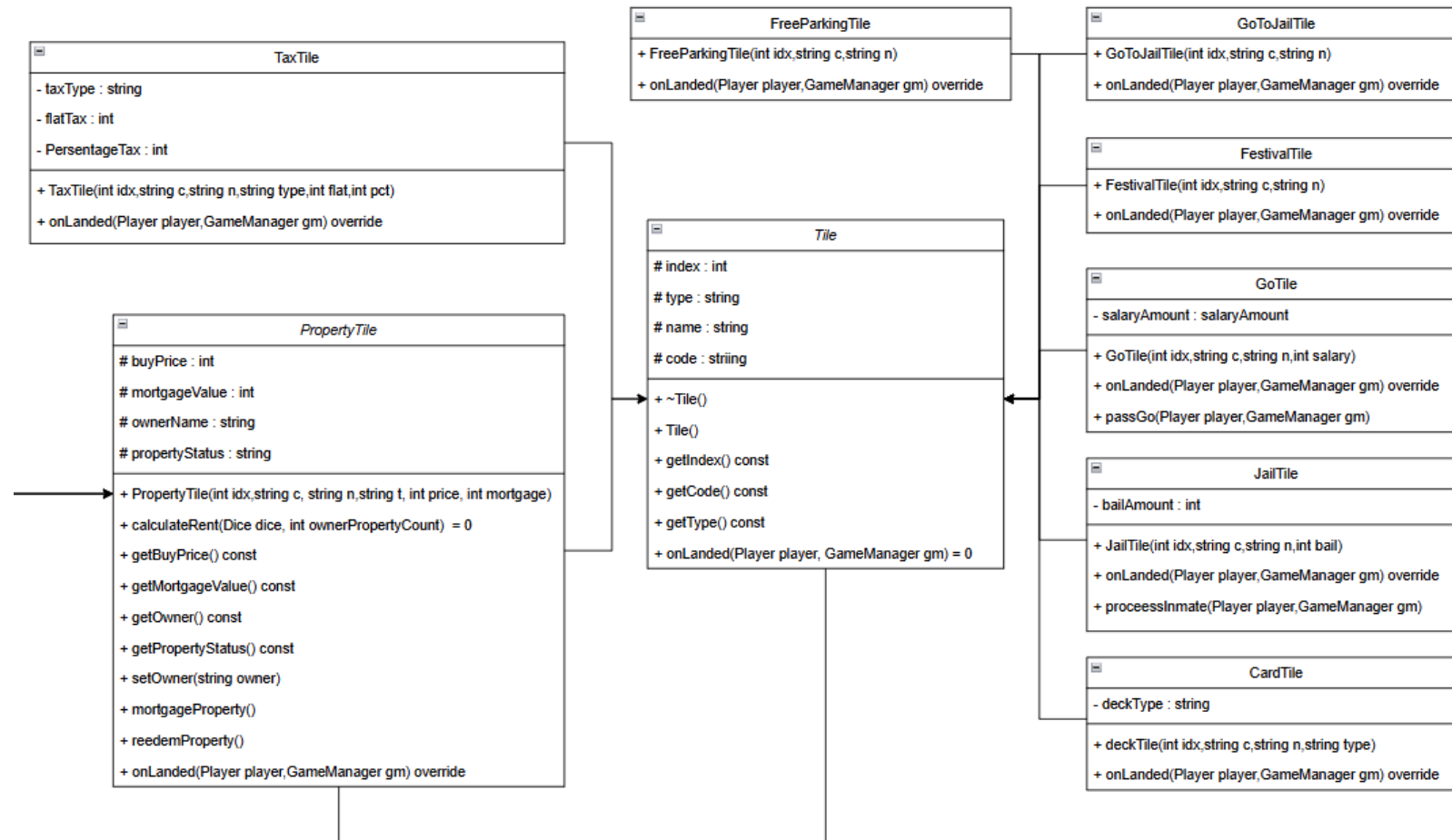
Nama Kelompok : **Nama Kelompok**

1. 13524119 - Nathanael Shane Bennet
2. 13524130 - Faris Wirakusuma Triawan
3. 13524144 - Jonathan Harijadi
4. 13524147 - Muh. Hartawan Haidir

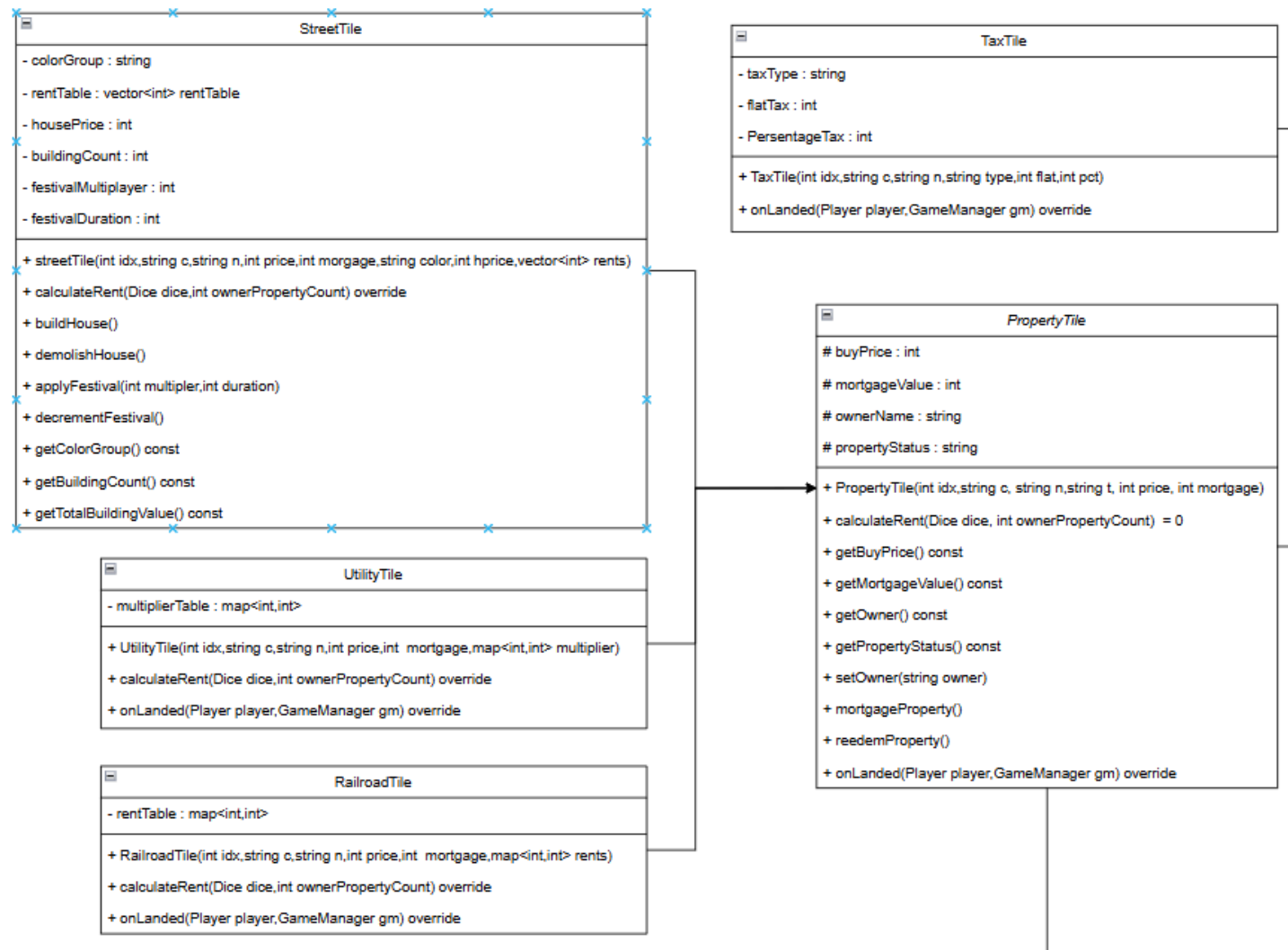


<Kode Kelompok> - <Nama Kelompok>

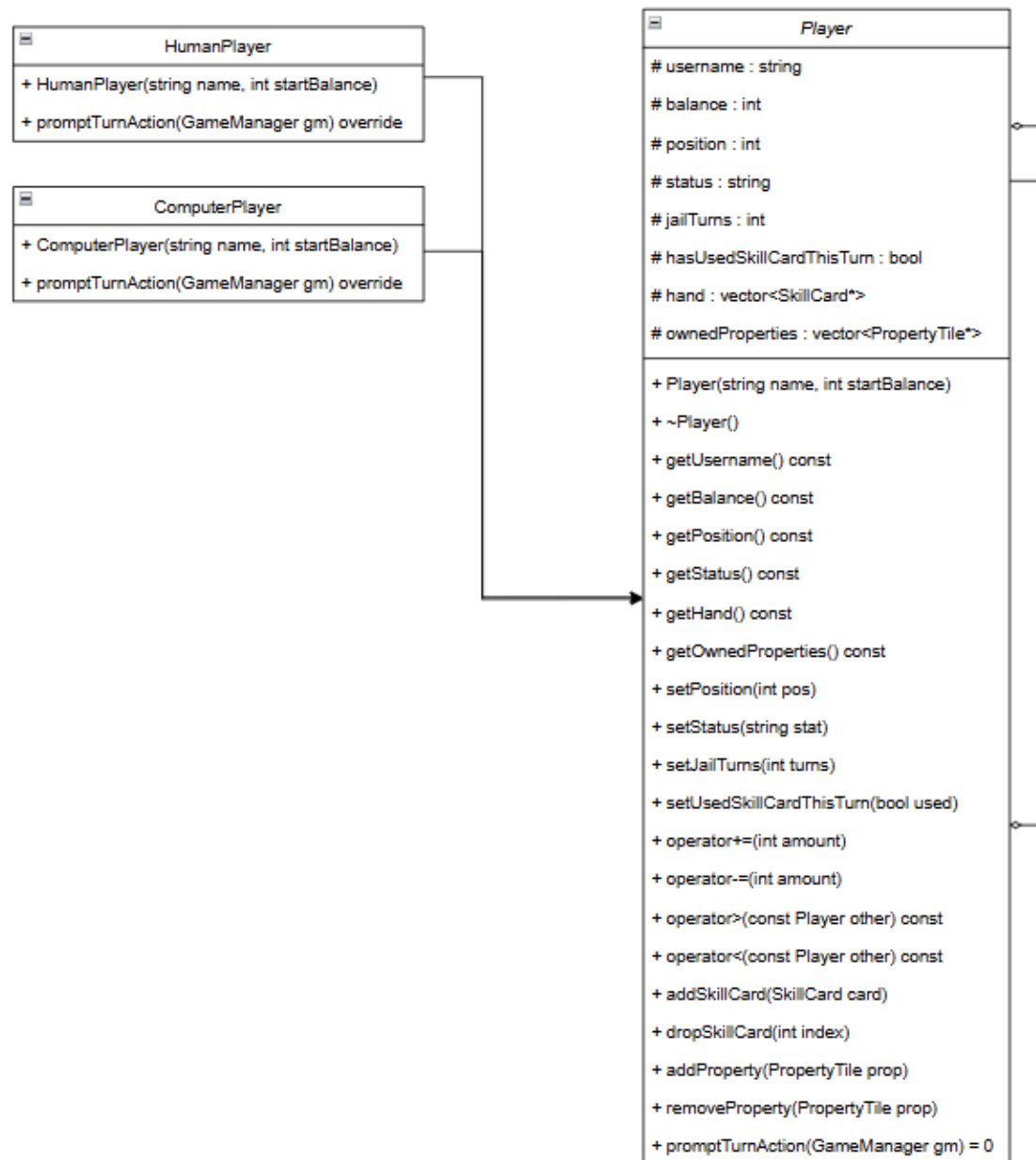
1. Diagram Kelas



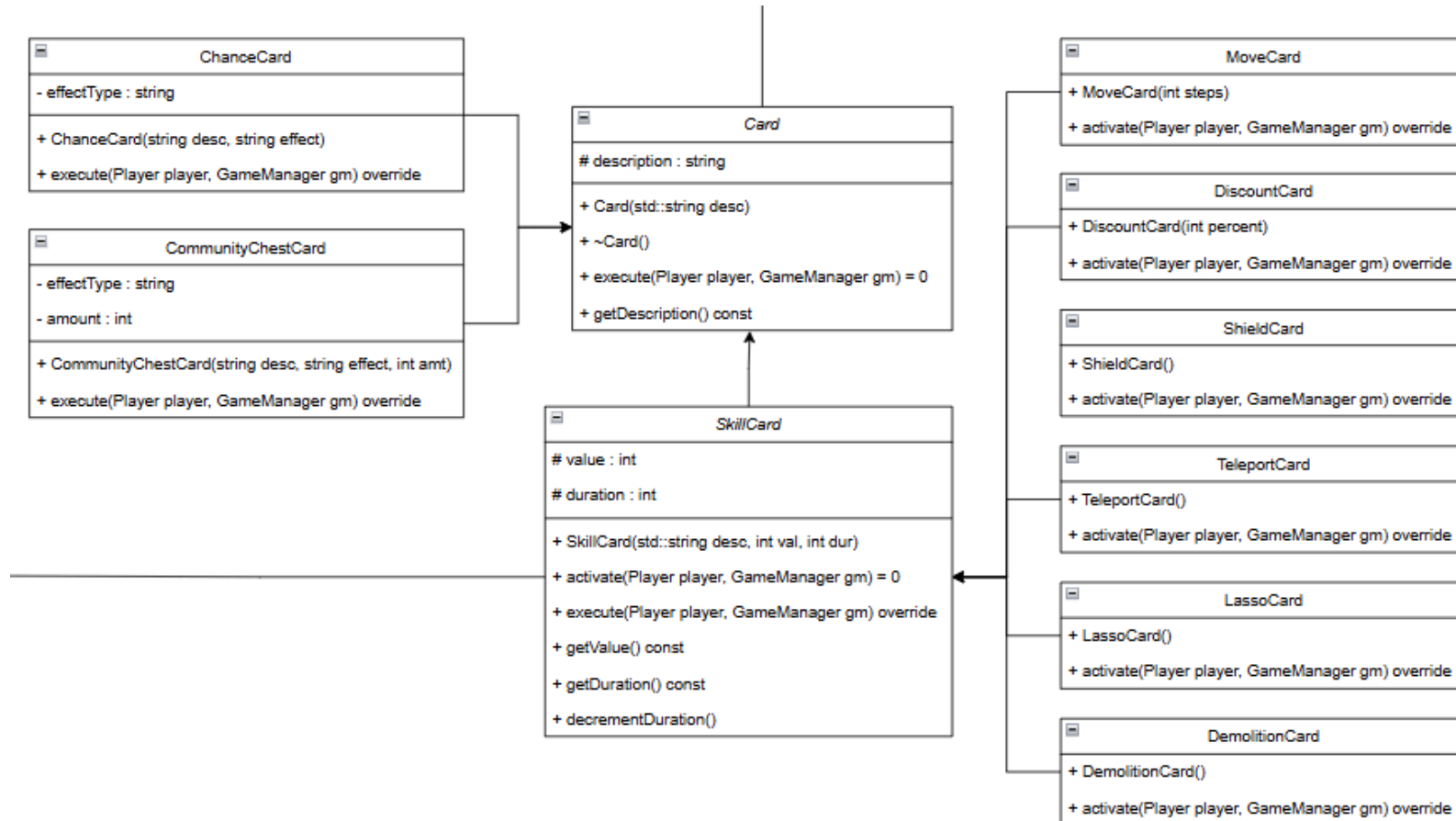
Gambar 1.1. Diagram Kelas Tile



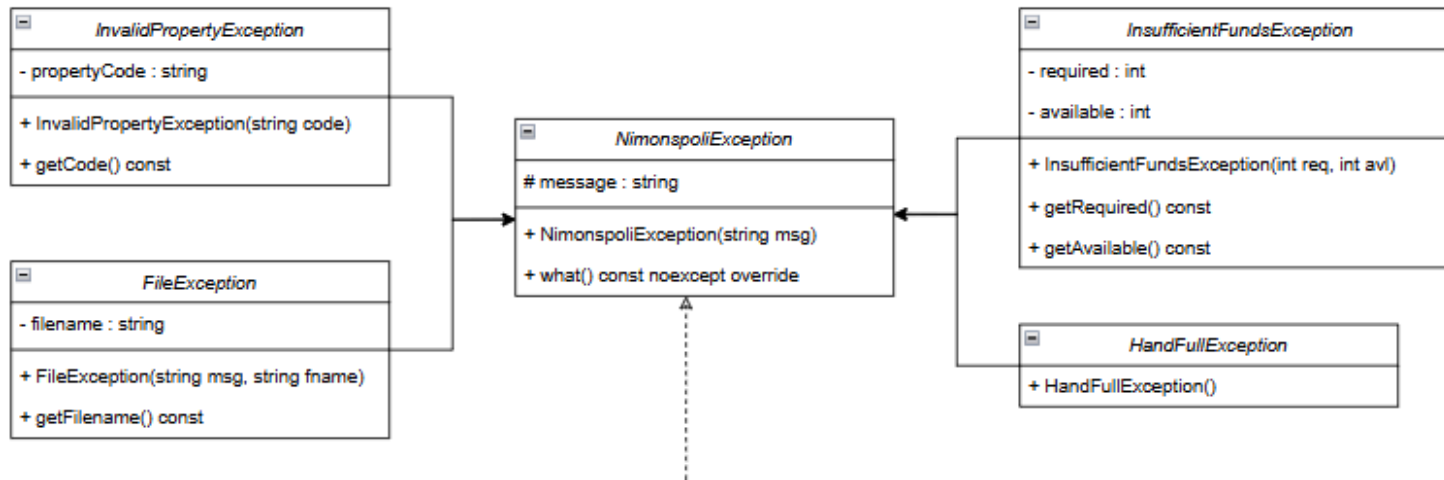
Gambar 1.2. Diagram Kelas Property Tile



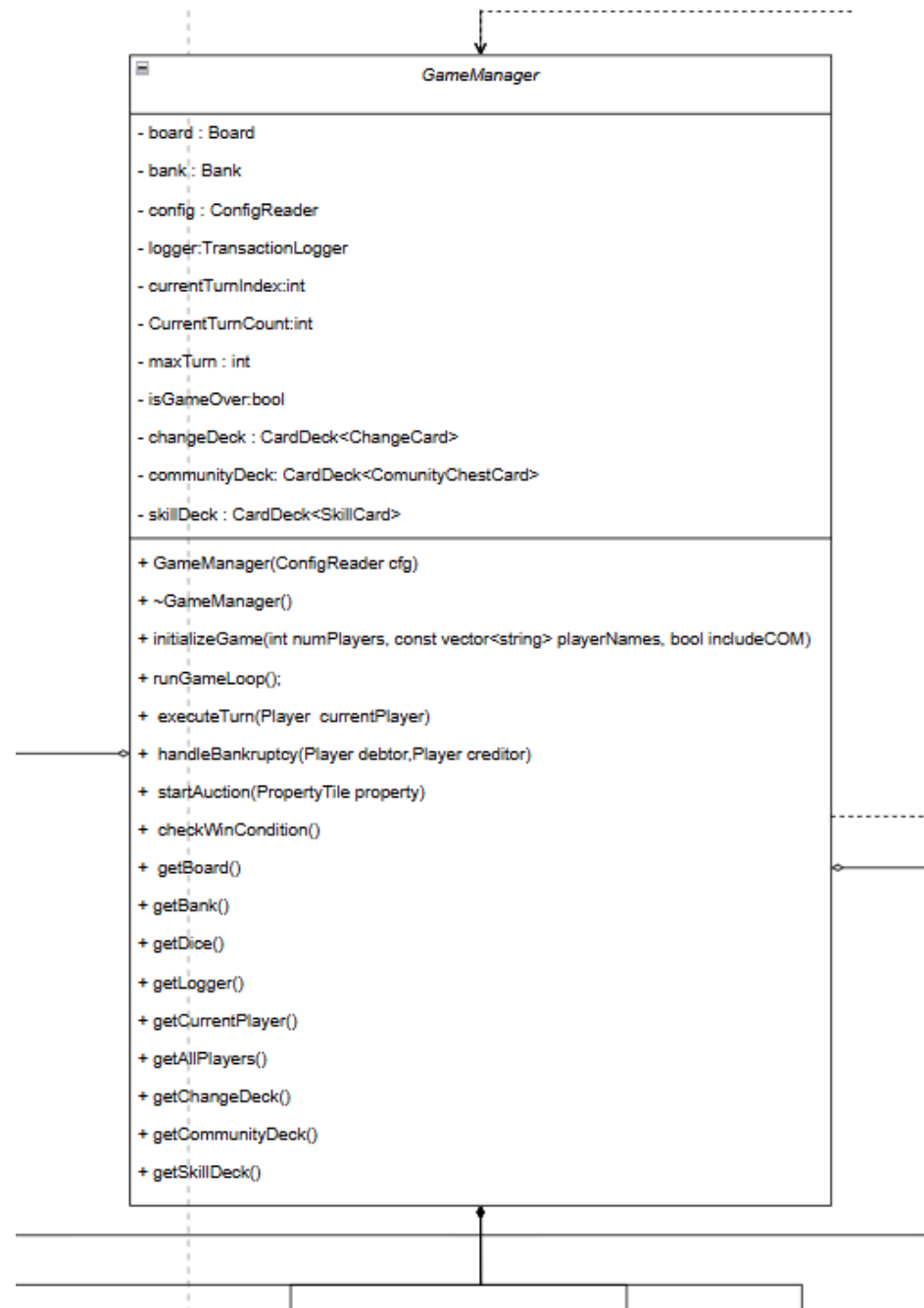
Gambar 1.3. Diagram Kelas Player



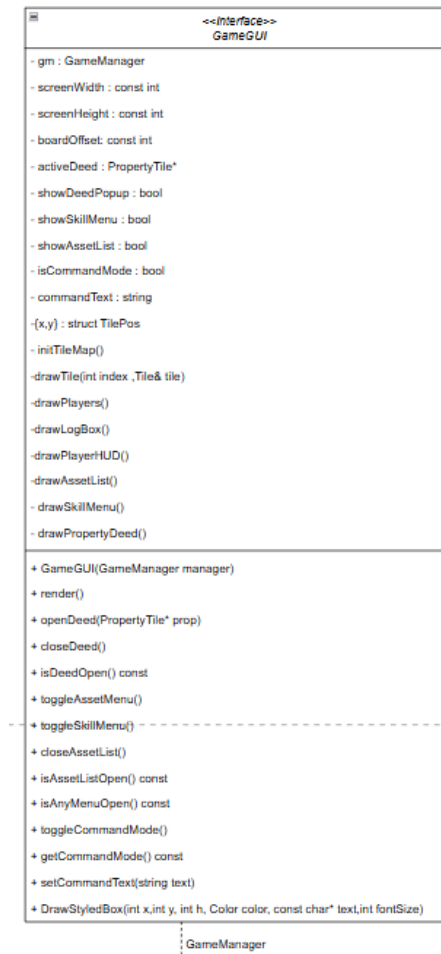
Gambar 1.4. Diagram Kelas Card



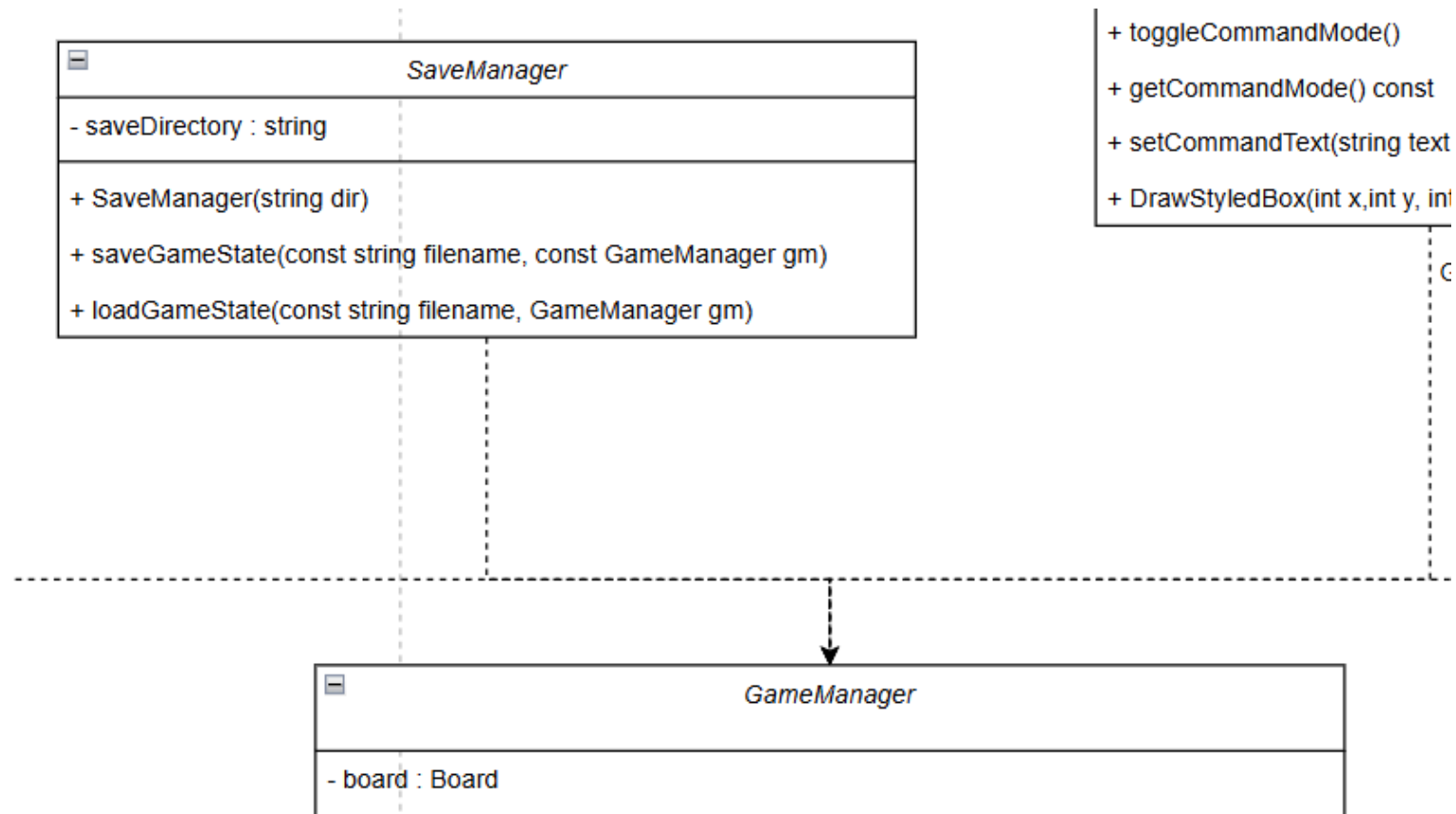
Gambar 1.5. Diagram Kelas Exception



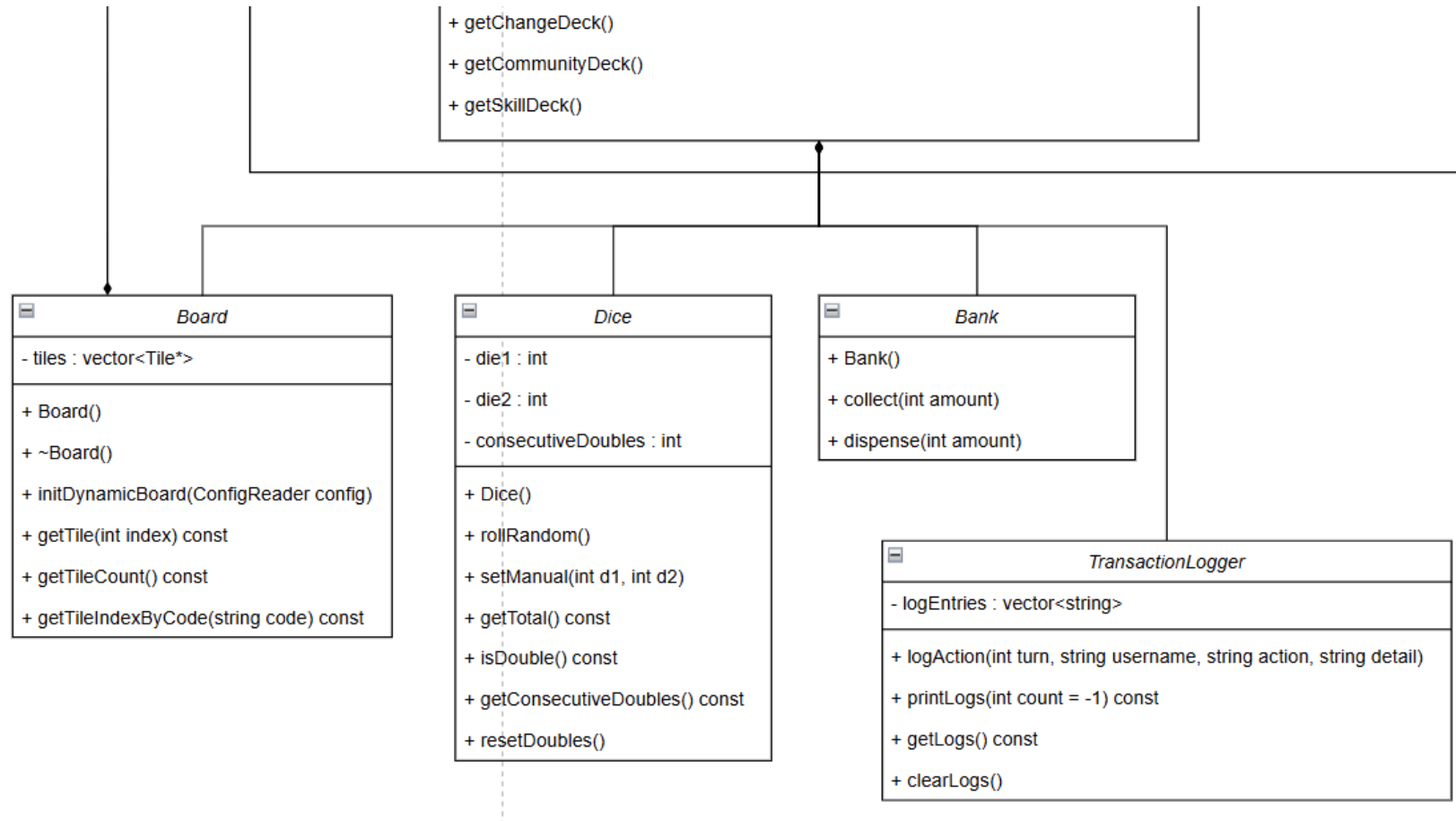
Gambar 1.6. Diagram Kelas GameManagerer



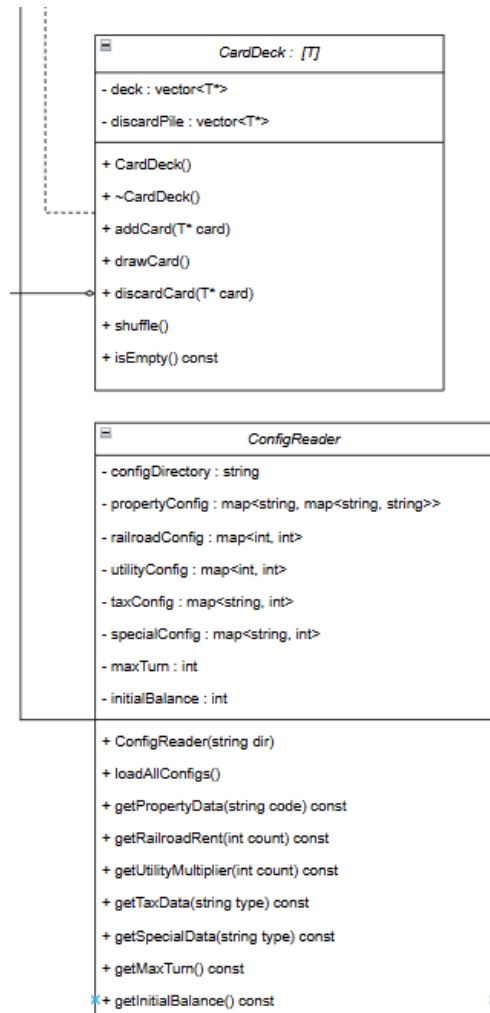
Gambar 1.7. Diagram Kelas GameCLI



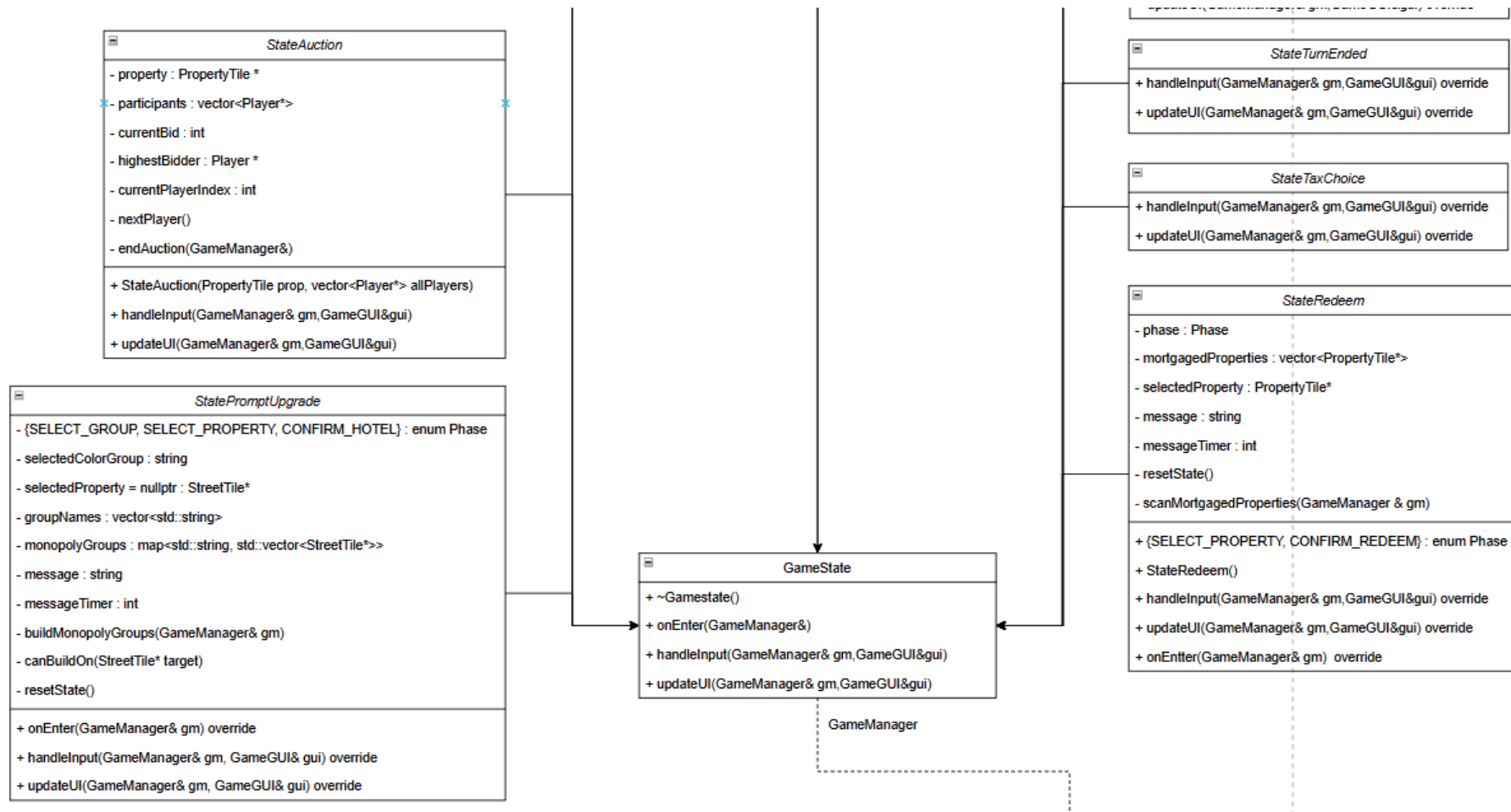
Gambar 1.8. Diagram Kelas SaveManager



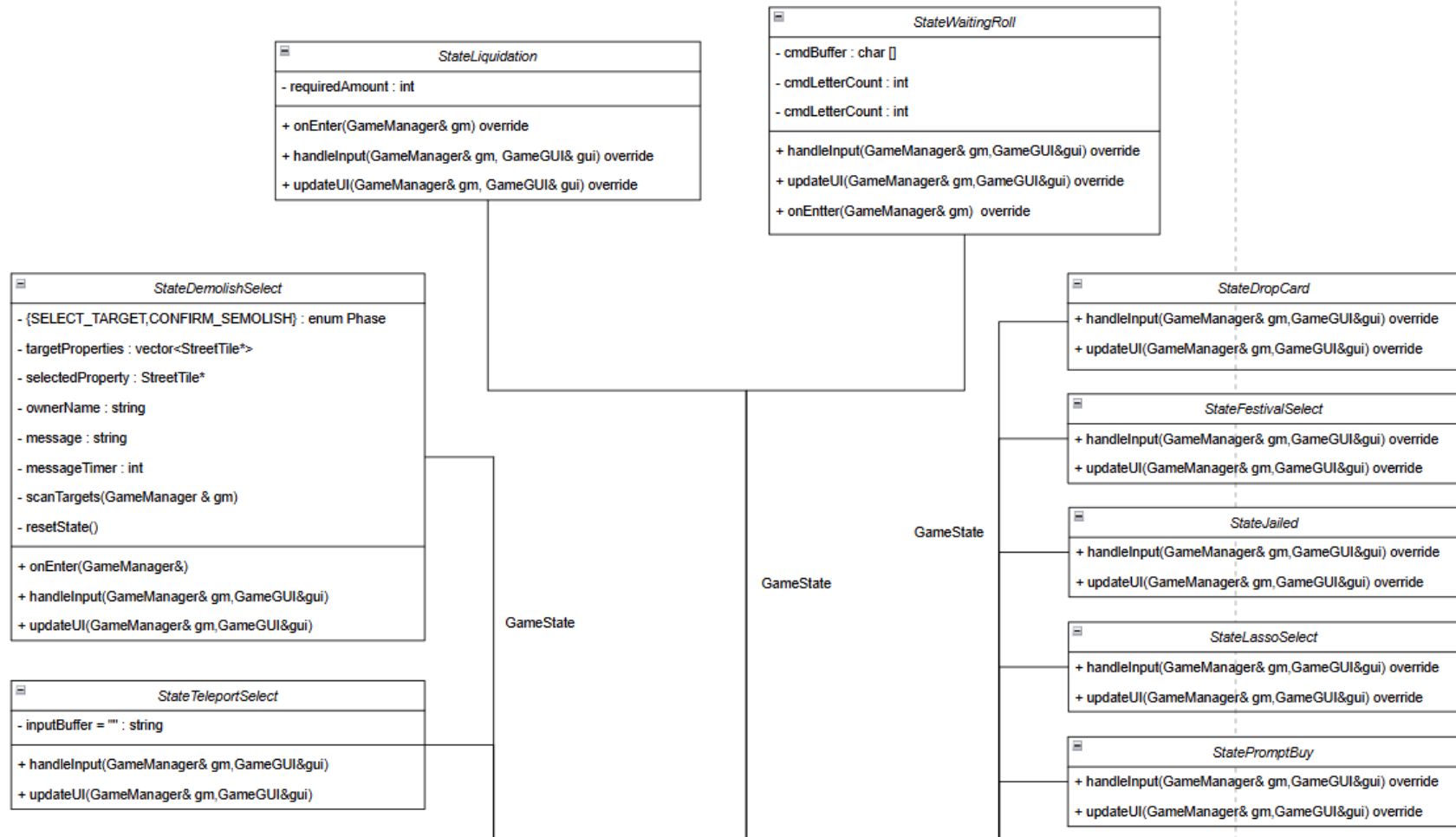
Gambar 1.9. Diagram Kelas Board, Dice, Bank, dan TransactionLogger



Gambar 1.10. Diagram Kelas CardDeck dan ConfigReader



Gambar 1.11. Diagram Kelas GameStateHierarchy 1



Gambar 1.12. Diagram Kelas GameStateHierarchy 2

2. Deskripsi Kelas

2.1. Kelas Exception Handling

2.1.1. Class NimospoliException : public std::exception

Kelas NimospoliException adalah pewaris dari kelas exception yang menjadi kelas dasar untuk semua exception yang mungkin terjadi di dalam permainan Nimonspoli

Nama Atribut	Return Type	Deskripsi
message	std::string	Pesan error yang mendeskripsikan penyebab exception; diisi oleh subclass melalui constructor chaining dengan initializer list

Nama Method	Return Type	Deskripsi
NimospolicException(msg)	constructor	Memanggil parent NimonspolicException dengan pesan 'Uang tidak cukup' dan menyimpan kedua nilai via initializer list
what()	Const char*	Override dari std::exception::what(); mengembalikan message.c_str() agar kompatibel dengan standar C++ exception handling

2.1.2. Class InsufficientFundsException : public NimospoliException

Kelas InsufficientFundsException akan dilempar saat pemain tidak memiliki cukup uang untuk membeli properti, membayar sewa, atau melakukan bid lelang.

Nama Atribut	Return Type	Deskripsi
<code>required</code>	<code>int</code>	Jumlah uang yang dibutuhkan untuk transaksi (misal: harga beli properti, nilai sewa yang harus dibayar)
<code>available</code>	<code>int</code>	Jumlah uang yang saat ini dimiliki pemain

Nama Method	Return Type	Deskripsi
<code>InsufficientFundsException(required, available)</code>	<code>constructor</code>	Menerima string pesan dan menyimpannya ke atribut message
<code>getRequired()</code>	<code>int</code>	Getter untuk jumlah yang dibutuhkan; berguna agar UI dapat menampilkan kekurangan secara detail
<code>getAvailable()</code>	<code>int</code>	Getter untuk jumlah yang tersedia; berguna untuk menghitung potensi likuidasi saat masuk alur kebangkrutan

2.1.3. Class HandFullException : public NimoSpoliException

Kelas HandFullException akan dilempar saat pemain mencoba menerima kartu ke-4 padahal tangan sudah penuh (maksimal 3 kartu)

Nama Method	Return Type	Deskripsi
HandFullException	constructor	Memanggil parent dengan pesan tetap 'kartu di tangan sudah penuh (maks 3).' tidak membutuhkan parameter tambahan karena batas kartu bersifat konstan

2.1.4. Class InvalidPropertyException : public NimoSpoliException

Kelas InvalidPropertyException akan dilempar saat properti tidak valid atau tidak ditemukan

Nama Atribut	Return Type	Deskripsi
propertyCode	std::string	Kode properti yang tidak valid atau tidak ditemukan (misal" 'XYZ'); disimpan untuk keperluan pesan error yang spesifik

Nama Method	Return Type	Deskripsi
<code>InvalidPropertyException (code)</code>	constructor	Memanggil parent dengan pesan 'Properti tidak ditemukan: <code>' dan menyimpan kode properti ke atribut <code>propertyCode</code>

2.1.5. Class `InvalidActionException` : `public NimoSpoliException`

Kelas `InvalidActionException` akan dilempar saat pemain melakukan aksi yang tidak valid, seperti membangun di properti yang tidak memenuhi syarat.

Nama Method	Return Type	Deskripsi
<code>InvalidActionException (action)</code>	constructor	Memanggil parent dengan pesan 'Aksi tidak valid: <action>' untuk mendeskripsikan aksi spesifik yang dilanggar, action bisa berupa nama perintah atau keterangan kondisi

2.1.6. Class `FileException` : `public NimospoliException`

Kelas `FileException` akan dilempar saat file yang akan diproses telah rusak (untuk save/Load) atau tidak ditemukan

Nama Method	Return Type	Deskripsi
<code>FileException (filename)</code>	constructor	Memanggil parent dengan pesan 'File error: <filename>' untuk mengidentifikasi file yang

		bermasalah; filename berisi nama atau path file yang gagal diakses
--	--	--

2.2. Kelas Game Logic

2.2.1. Class GameManager

GameManager adalah class utama yang bertanggungjawab sebagai koordinator utama dalam mengontrol alur permainan Nimonspoli dari awal sampai akhir. Kelas ini menerapkan *Single Responsibility Principle* dengan hanya berfokus pada manajemen giliran dan kondisi menang/kalah. Berikut adalah penjelasan atribut dan method untuk `class GameManager` :

Nama Atribut	Return Type	Deskripsi
board	Board	Papan permainan yang berisi semua Tile
bank	Bank	Objek bank permainan
dice	Dice	Objek dadu permainan
logger	TransactionLogger	Logger atau kertas untuk mencatat semua transaksi
config	ConfigReader	Pointer ke konfigurasi yang dimuat saat startup
players	<code>std::vector<Player*></code>	Pointer ke daftar semua pemain (aktif maupun bangkrut)

<code>currentTurnIndex</code>	<code>int</code>	Indeks pemain yang sedang mendapat giliran
<code>currentTurnCount</code>	<code>int</code>	Hitungan giliran yang telah berjalan
<code>maxTurn</code>	<code>int</code>	Batas maksimum giliran dari konfigurasi
<code>isGameOver</code>	<code>bool</code>	Flag Penanda Permainan sudah selesai
<code>chanceDeck</code>	<code>CardDeck<ChanceCard></code>	Tumpukan kartu kesempatan
<code>communityDeck</code>	<code>CardDeck<CommunityChestCard></code>	Tumpukan kartu Dana Umum
<code>skillDeck</code>	<code>CardDeck<SkillCard></code>	Tumpukan kartu Kemampuan Spesial

Nama Method	Return Type	Deskripsi
<code>GameManager (ConfigReader cfg)</code>	<code>constructor</code>	Menerima pointer ConfigReader dan menginisialisasi semua komponen
<code>~GameManager ()</code>	<code>destructor</code>	Membebaskan memori semua Player dan ConfigReader
<code>initializeGame (int numPlayers, const std::vector<std::string>& playerNames, bool includeCOM)</code>	<code>void</code>	Membuat pemain, mengocok urutan giliran, dan menyiapkan deck kartu

<code>runGameLoop()</code>	<code>void</code>	Loop utama permainan : memanggil <code>executeTurn()</code> berulang hingga kondisi selesai
<code>executeTurn(Player& currentPlayer)</code>	<code>void</code>	Menjalankan satu giliran pemain : distribusi kartu, aksi, dan efek petak
<code>handleBankruptcy(Player& debtor, Player* creditor)</code>	<code>void</code>	Menangani alur kebangkrutan : likuidasi aset, pengalihan ke kreditor/bank, lelang
<code>startAuction(PropertyTile& property)</code>	<code>void</code>	Menjalankan mekanisme lelang properti sesuai spesifikasi
<code>checkWinCondition()</code>	<code>void</code>	Mengecek kondisi menang (<code>MAX_TURN</code> atau <code>BANKRUPTCY</code>) dan mengakhiri permainan
<code>getBoard()</code>	<code>Board&</code>	Getter Board
<code>getBank()</code>	<code>Bank&</code>	Getter Bank
<code>getDice()</code>	<code>Dice&</code>	Getter Dice
<code>getLogger()</code>	<code>TransactionLogger&</code>	Getter Logger
<code>getCurrentPlayer()</code>	<code>Player&</code>	Getter pemain yang sedang mendapat giliran
<code>getAllPlayers()</code>	<code>std::vector<Player*>&</code>	Getter semua pemain
<code>getChanceDeck()</code>	<code>CardDeck<ChanceCard>&</code>	Getter deck Kesempatan
<code>getCommunityDeck()</code>	<code>CardDeck<CommunityChestCard>&</code>	Getter deck Dana Umum

<code>getSkillDeck()</code>	<code>CardDeck<SkillCard>&</code>	Getter Kemampuan Spesial
-----------------------------	---	--------------------------

2.2.2. Class Bank

Kelas Bank adalah kelas utama yang merepresentasikan bank di kehidupan nyata dalam permainan Monopoli. Kelas ini akan mengatur aliran keuangan yang masuk dan yang keluar kepada pemain.

Nama Method	Return Type	Deskripsi
<code>Bank()</code>	<code>constructor</code>	Menginisialisasi bank
<code>collect(int amount)</code>	<code>void</code>	Menerima sejumlah uang dari pemain (pajak, gadai, dsb)
<code>dispense(int amount)</code>	<code>void</code>	Mengeluarkan sejumlah uang kepada pemain (gaji GO, tebus, dsb)

2.2.3. Class Dice

Kelas Dice adalah kelas utama yang merepresentasikan dua dadu dalam permainan. Kelas ini akan menangani lemparan acak, pengaturan manual, dan deteksi double (termasuk triple double yang mengirim pemain ke penjara)

Nama Atribut	Return Type	Deskripsi
<code>die1</code>	<code>int</code>	Nilai dadu pertama (1-6)

die2	int	Nilai dadu kedua (1-6)
consecutiveDoubles	int	Jumlah double berturut-turut dalam satu giliran

Nama Method	Return Type	Deskripsi
Dice()	constructor	Menginisialisasi semua ke nilai 0
rollRandom	void	Mengacak nilai die1 dan die2 secara random
getManual(int d1, int d2)	void	Mengatur nilai die1 dan die2 secara manual melalui perintah (ATUR DADU)
getTotal() const	int	Mengembalikan nilai die1+die2
isDouble() const	bool	Mengembalikan true jika die1==die2
getConsecutiveDoubles() const	int	Getter untuk jumlah double berturut turut
resetDoubles()	void	Mereset consecutive Double ke 0 (Dipanggil saat giliran berakhir normal)

2.2.4. Class Player

Kelas Player adalah kelas utama yang merepresentasikan pemain (baik itu human ataupun komputer) dalam kehidupan nyata. Kelas ini akan menyimpan state pemain (uang, posisi, status, kartu, properti).

Nama Atribut	Return Type	Deskripsi
username	std::string	Nama unik pemain
balance	int	Jumlah uang tunai yang dimiliki pemain saat ini
position	int	Indek petak papan tempat bidak pemain berada (1-40)
status	std::string	Status pemain: 'ACTIVE', 'BANKRUPT', atau 'JAILED'
jailTurns	int	Jumlah giliran pemain sudah berada di penjara(0 jika tidak dipenjara)
hasUsedSkillCardThisTurn	bool	Flag untuk membatasi penggunaan kartu kemampuan maksimal 1x per giliran
hand	std::vector<SkillCard*>	Daftar kartu kemampuan di tangan pemain (maks 3 kartu)
ownedProperties	std::vector<PropertyTile*>	Daftar properti yang dimiliki pemain

Nama Method	Return Type	Deskripsi
<code>Player(std::string name, int startBalance)</code>	constructor	Menginisialisasi pemain dengan nama dan saldo awal
<code>~Player()</code>	Destructor (virtual)	Menghapus memori dari player
<code>getUsername() const</code>	<code>std::string</code>	Getter username
<code>getBalance() const</code>	<code>int</code>	Getter saldo uang tunai
<code>getPosition() const</code>	<code>int</code>	Getter posisi bidak
<code>getStatus() const</code>	<code>std::string</code>	Getter status pemain
<code>getTotalWealth() const</code>	<code>int</code>	Menghitung total kekayaan: uang + harga beli properti + nilai bangunan
<code>getHand() const</code>	<code>const std::vector<SkillCard*>&</code>	Getter daftar kartu di tangan
<code>getOwnedProperties() const</code>	<code>const std::vector<PropertyTile*>&</code>	Getter daftar properti yang dimiliki
<code>setPosition(int pos)</code>	<code>void</code>	Mengatur posisi dari pemain
<code>setStatus(std::string stat)</code>	<code>void</code>	Pengatur status dari pemain
<code>setJailTurns(int turns)</code>	<code>void</code>	Mengatur giliran dari pemain untuk menuju ke penjara
<code>setUsedSkillCardThisTurn(bool used)</code>	<code>void</code>	Mengatur giliran skill card dari pemain
<code>operator+=(int amount)</code>	<code>Player&</code>	Overload +=: menambah saldo pemain (menerima uang/gaji)

<code>operator--(int amount)</code>	<code>Player&</code>	Overload -=: mengurangi saldo pemain (membayar sewa/pajak)
<code>operator>(const Player& other) const</code>	<code>bool</code>	Overload >: membandingkan total kekayaan antar pemain (untuk klasemen akhir)
<code>operator<(const Player& other) const</code>	<code>bool</code>	Overload <: membandingkan total kekayaan antar pemain
<code>addSkillCard(SkillCard* card)</code>	<code>void</code>	Menambahkan kartu ke tangan; melempar <code>HandFullException</code> jika sudah 3 kartu
<code>dropSkillCard(int index)</code>	<code>void</code>	Membuang kartu pada indeks tertentu dari tangan
<code>addProperty(PropertyTile* prop)</code>	<code>void</code>	Menambahkan properti dari daftar kepemilikan pemain
<code>removeProperty(PropertyTile* prop)</code>	<code>void</code>	Menghapus properti dari daftar kepemilikan pemain
<code>promptTurnAction(GameManager& gm) = 0</code>	<code>void(pure virtual)</code>	Menangani aksi giliran, implementasi berbeda oleh Human vs COM

2.2.5. Class `HumanPlayer` : public `Player`

Kelas `HumanPlayer` adalah pewaris dari Kelas `Player` yang merepresentasikan pemain manusia. Kelas ini akan menampilkan menu dan membaca input dari terminal

Nama Method	Return Type	Deskripsi
<code>HumanPlayer(std::string name, int startBalance)</code>	constructor	Menginisialisasi atribut dari human player
<code>promptTurnAction(GameManager& gm) override</code>	void	Melakukan aksi yang telah ditentukan

2.2.6. Class ComputerPlayer : public Player

Kelas ComputerPlayer adalah pewaris dari Kelas player yang merepresentasikan pemain komputer. Kelas ini akan menjalankan aksi otomatis berdasarkan strategi sederhana.

Nama Method	Return Type	Deskripsi
<code>ComputerPlayer(std::string name, int startBalance)</code>	constructor	Menginisialisasi atribut dari player komputer
<code>promptTurnAction(GameManager& gm) override</code>	void	Melakukan aksi yang telah ditentukan

2.2.7. Class Board

Kelas Board adalah kelas utama yang merepresentasikan papan permainan berisi 40 petak (atau jumlah dinamis untuk bonus papan dinamis). Menginisialisasi dan menyimpan semua Tile

Nama Atribut	Return Type	Deskripsi
<code>tiles</code>	<code>std::vector<T*></code>	Koleksi semua petak papan secara berurutan sesuai indeks

Nama Method	Return Type	Deskripsi
<code>Board()</code>	constructor	Menginisialisasi vektor tiles kosong
<code>~Board()</code>	Destructor	Membebaskan memori semua tile
<code>initDynamicBoard(ConfigReader& config)</code>	void	Membangun papan dari konfigurasi; mendukung papan dinamis (bonus)
<code>getTile(int index) const</code>	<code>Tile&</code>	Mengembalikan referensi ke Tile pada indeks tertentu
<code>getTileCount() const</code>	<code>int</code>	Mengembalikan jumlah total petak di papan
<code>getTileIndexByCode(std::string code) const</code>	<code>int</code>	Mencari dan mengembalikan indeks petak berdasarkan kode (misal: 'JKT')

2.3. Kelas Generic & Card Hierarchy

2.3.1. Class CardDeck

Template class generik untuk mengelola berbagai tumpukan kartu (Chance, Community Chest, SkillCard). Menangani draw, discard, dan shuffle secara seragam untuk semua tipe kartu.

Nama Atribut	Return Type	Deskripsi
deck	<code>std::vector<T*></code>	Tumpukan kartu aktif yang siap diambil pemain
discardPile	<code>std::vector<T*></code>	Tumpukan kartu yang telah digunakan; dikocok ulang saat deck habis

Nama Method	Return Type	Deskripsi
<code>CardDeck()</code>	constructor	Menginisialisasi deck dan <code>discardPile</code> sebagai vector kosong
<code>~CardDeck()</code>	Destructor	Membebaskan memori semua pointer kartu yang dimiliki
<code>addCard(T* card)</code>	void	Menambahkan kartu ke dalam deck
<code>drawCard()</code>	<code>T*a</code>	Mengambil kartu teratas dari deck; jika deck kosong, reshuffle dari <code>discardPile</code>
<code>discardCard(T* card)</code>	void	Memindahkan kartu ke <code>discardPile</code> setelah digunakan
<code>shuffle()</code>	void	Mengocok urutan kartu dalam deck secara acak
<code>isEmpty() const</code>	bool	Mengembalikan true jika deck kosong

2.3.2. Class Card

Kelas Card adalah kelas abstrak utama yang mnejadi kelas untuk semua jenis kartu (Chance, Community Chest, SkillCard). Menyediakan interface execute() yang harus diimplementasi oleh setiap subclass

Nama Atribut	Return Type	Deskripsi
description	std::string	Teks deskripsi efek kartu yang ditampilkan ke pemain

Nama Method	Return Type	Deskripsi
Card(std::string desc)	constructor	Menerima string deskripsi kartu
~Card()	Destructor (virtual)	Menghapus string kartu
execute(Player& player, GameManager& gm)	void(pure virtual)	Menjalankan efek kartu terhadap pemain dan state permainan
getDescription() const	std::string	Mengembalikan deskripsi kartu untuk ditampilkan di UI

2.3.3. Class ChanceCard : public Card

Kelas ChanceCard adalah pewaris dari kelas Card yang merepresentasikan kartu kesempatan. Efek yang tersedia adalah pergi ke stasiun terdekat, mundur 3 petak, atau masuk penjara

Nama Atribut	Return Type	Deskripsi
effectType	std::string	Tipe efek kartu (misal: 'NEAREST_STATION', BACK_3', GO TO JAIL')

Nama Method	Return Type	Deskripsi
ChanceCard(std::string desc, std::string effect)	constructor	Menerima deskripsi dan tipe efek
execute(Player& player, GameManager& gm) override	void	Mengeksekusi efek sesuai effectType terhadap pemain

2.3.4. Class CommunityChestCard : public Card

Kelas CommunityChestCard adalah pewaris dari kelas Card yang merepresentasikan kartu Dana Umum. Efek mencakup menerima uang dari pemain lain (ulang tahun), membayar ke bank (biaya dokter), atau membayar ke semua pemain (nyaleg)

Nama Atribut	Return Type	Deskripsi
effectType	std::string	Tipe efek('BIRTHdaY', 'DOCTOR_FEE', 'CAMPAIGN')
amount	int	Jumlah uang yang terlibat dalam efek kartu

Nama Method	Return Type	Deskripsi
CommunityChestCard(std::string desc, std::string effect, int amt)	constructor	Menerima deskripsi, tipe efek, dan nominal uang
execute(Player& player, GameManager& gm) override	void	Mengeksekusi efek transfer uang sesuai tipe kartu

2.3.5. Class SkillCard : public Card

Kelas SkillCard adalah pewaris dari kelas card yang juga merupakan kelas abstrak untuk kartu kemampuan Spesial. Menyimpan nilai dan durasi berlakunya kartu. Subclass harus mengimplementasikan active() sesuai jenis kemampuannya.

Nama Atribut	Return Type	Deskripsi
value	int	Nilai numerik kartu(misal: jumlah langkah MoveCard, %diskon DiscountCard)

<code>duration</code>	<code>int</code>	Sisa durasi berikutnya kartu dalam giliran
-----------------------	------------------	--

Nama Method	Return Type	Deskripsi
<code>SkillCard(std::string desc, int val, int dur)</code>	constructor	Menerima deskripsi, nilai, dan durasi awal
<code>activate(Player& player, GameManager& gm) override</code>	<code>void(pure virtual)</code>	Mengaktifkan efek khusus kartu; harus diimplementasi subclass
<code>execute(Player& player, GameManager& gm) override</code>	<code>void</code>	Override dari <code>Card::execute()</code> ; memanggil <code>activate()</code>
<code>getValue() const</code>	<code>int</code>	Getter untuk nilai kartu
<code>getDuration() const</code>	<code>int</code>	Getter untuk sisa durasi
<code>decrementDuration()</code>	<code>void</code>	Mengurangi sisa durasi sebesar 1 (dipanggil setiap awal giliran pemain)

2.3.6. Class MoveCard : public SkillCard

Kelas MoveCard adalah pewaris dari kelas SkillCard yang memiliki peran untuk memindahkan pemain maju sejumlah petak (nilai ditentukan acak saat kartu didapat)

Nama Method	Return Type	Deskripsi
<code>MoveCard(int steps)</code>	<code>constructor</code>	Menginisialisasi kelas movecard
<code>activate(Player& player, GameManager& gm) override</code>	<code>void</code>	Mengaktifkan skill milik card

2.3.7. Class DiscountCard : public SkillCard

Kelas DiscountCard(percent) adalah pewaris dari kelas SkillCard yang memiliki peran untuk memberikan diskon % acak untuk 1 giliran. Value = persentase diskon

Nama Method	Return Type	Deskripsi
<code>DiscountCard(int percent)</code>	<code>constructor</code>	Menginisialisasi kelas discountcard
<code>activate(Player& player, GameManager& gm) override</code>	<code>void</code>	Mengaktifkan skill milik card

2.3.8. Class ShieldCard : public SkillCard

Kelas ShieldCard adalah pewaris dari kelas SkillCard yang memiliki peran untuk melindungi pemain dari semua tagihan/sanksi selama 1 giliran

Nama Method	Return Type	Deskripsi
<code>ShieldCard()</code>	<code>constructor</code>	Menginisialisasi kelas shieldcard
<code>activate(Player& player, GameManager& gm) override</code>	<code>void</code>	Mengaktifkan skill milik card

2.3.9. Class TeleportCard : public SkillCard

Kelas TeleportCard adalah pewaris dari kelas SkillCard yang memiliki peran untuk memungkinkan pemain berpindah ke petak manapun di papan

Nama Method	Return Type	Deskripsi
<code>TeleportCard()</code>	<code>constructor</code>	Menginisialisasi kelas teleportCard
<code>activate(Player& player, GameManager& gm) override</code>	<code>void</code>	Mengaktifkan skill milik card

2.3.10. Class LassoCard : public SkillCard

Kelas LassoCard adalah pewaris dari kelas SkillCard yang memiliki peran untuk menarik satu pemain lawan yang berada di depan ke posisi pemain saat ini

Nama Method	Return Type	Deskripsi
<code>LassoCard()</code>	<code>constructor</code>	Menginisialisasi kelas LassoCard
<code>activate(Player& player, GameManager& gm) override</code>	<code>void</code>	Mengaktifkan skill milik card

2.3.11. Class DemoLitionCard : public SkillCard

Kelas DemoLitionCard adalah pewaris dari kelas SkillCard yang memiliki peran untuk menghancurkan satu bangunan di properti milik pemain lawan

Nama Method	Return Type	Deskripsi
<code>DemoLitionCard()</code>	<code>constructor</code>	Menginisialisasi kelas DemoLitionCard
<code>activate(Player& player, GameManager& gm) override</code>	<code>void</code>	Mengaktifkan skill milik card

2.3.12. Class GameState

Kelas **GameState** adalah *Abstract Base Class* (kelas dasar abstrak) yang menjadi fondasi dari *State Pattern* dalam antarmuka permainan. Kelas ini memisahkan logika antarmuka dan kontrol permainan berdasarkan fase atau kejadian tertentu (misal: saat mau beli tanah, saat lelang, atau saat giliran berakhir), sehingga *game loop* utama terbebas dari rantai *if-else* yang rumit.

Nama Method	Return Type	Deskripsi
<code>~GameState()</code>	<code>destructor</code>	Menghancurkan objek game state
<code>onEnter(GameManager&)</code>	<code>Virtual void</code>	Dieksekusi tepat satu kali ketika <i>state</i> baru saja dipanggil atau diaktifkan (berguna untuk inisialisasi variabel awal <i>state</i>)
<code>handleInput(GameManager& gm, GameGUI& gui)</code>	<code>Virtual void</code>	Menangani dan membaca input dari pemain (keyboard/mouse) yang valid dan relevan khusus untuk <i>state</i> saat ini.
<code>updateUI (GameManager& gm, GameGUI& gui)</code>	<code>Virtual void</code>	Menggambar elemen visual atau antarmuka (<i>pop-up</i> , teks, menu) ke layar sesuai dengan konteks <i>state</i> tersebut tanpa mengganggu <i>render</i> papan utama.

2.3.13. Class StatePromptBuy : public GameState

Kelas `StatePromptBuy` dipanggil ketika pemain mendarat di petak properti yang belum memiliki pemilik (milik BANK).

- **Fungsi Utama:** Menampilkan panel konfirmasi di layar yang menawarkan pemain untuk membeli properti tersebut sesuai harga beli, atau menolaknya (yang akan otomatis memicu pelelangan).

2.3.14. Class StatePromptUpgrade : public GameState

Kelas `StatePromptUpgrade` dipanggil ketika pemain mendarat di petak properti miliknya sendiri.

- **Fungsi Utama:** Mengecek level bangunan saat ini (maksimal Hotel / Level 5). Jika masih bisa dibangun, *state* ini akan menampilkan antarmuka konfirmasi untuk membangun/ *upgrade* rumah dengan memotong saldo pemain sesuai harga rumah di petak tersebut.

2.3.15. **Class StateAuction : public GameState**

Kelas *StateAuction* adalah *mini-game* lelang yang dipanggil saat seorang pemain menolak tawaran pembelian properti (dari *StatePromptBuy*).

- **Fungsi Utama:** Menahan alur giliran utama permainan dan membuat putaran giliran internal (khusus peserta lelang). Mengelola *array* pemain yang berpartisipasi, mencatat penawaran (*bid*) tertinggi, dan mengeliminasi pemain yang menekan tombol *Fold* (Nyerah) hingga tersisa satu pemenang.

2.3.16. **Class StateFestivalSelect : public GameState**

Kelas *StateFestivalSelect* dipanggil ketika pemain mendarat di petak *FestivalTile*.

- **Fungsi Utama:** Menampilkan layar instruksi yang meminta pemain untuk memilih salah satu properti miliknya untuk menyelenggarakan festival. Menerima input nomor properti, lalu menerapkan pengali ganda (*festivalMultiplier*) ke properti yang dipilih.

2.3.17. **Class StateTurnEnded : public GameState**

Kelas *StateTurnEnded* adalah *state* penutup atau transisi yang menandakan seluruh rangkaian aksi seorang pemain telah selesai.

- **Fungsi Utama:** Menampilkan animasi atau teks peringatan (*pop-up* di tengah papan) dari transaksi terakhir yang baru saja terjadi (seperti "BAYAR SEWA!", "MASUK PENJARA!", "PROPERTI DIBELI!") dan menunggu pemain menekan tombol *[ENTER]* sebelum memindahkan giliran ke pemain berikutnya.

2.4. Kelas Data Access Layer

2.4.1. Class ConfigReader

Kelas ConfigReader adalah kelas yang bertanggung jawab dalam membaca dan menyimpan seluruh data konfigurasi dari file eksternal (`property.txt`, `railroad.txt`, `utility.txt`, `tax.txt`, `special.txt`, `misc.txt`). Merupakan satu-satunya kelas yang berinteraksi dengan file konfigurasi

Nama Atribut	Return Type	Deskripsi
<code>configDirectory</code>	<code>std::string</code>	Path direktori tempat file konfigurasi berada
<code>propertyConfig</code>	<code>std::map<std::string, std::map<std::string, std::string>></code>	Menyimpan data semua properti, di-index berdasarkan kode properti
<code>railroadConfig</code>	<code>std::map<int, int></code>	Menyimpan faktor pengali sewa utility berdasarkan jumlah utility
<code>utilityConfig</code>	<code>std::map<int, int></code>	Menyimpan faktor pengali sewa utility berdasarkan jumlah utility
<code>taxConfig</code>	<code>std::map<std::string, int></code>	Menyimpan nilai pajak (PPH_FLAT, PPH_PERSENTASE, PBM_FLAT)
<code>specialConfig</code>	<code>std::map<std::string, int></code>	Menyimpan nilai petak spesial (GO_SALARY, JAIL_FINE)
<code>maxTurn</code>	<code>int</code>	Batas maksimum giliran permainan

		dari file misc.txt
initialBalance	int	Saldo awal setiap pemain dari file misc.txt

Nama Method	Return Type	Deskripsi
ConfigReader (std::string dir)	constructor	Menerima path direktor konfigurasi
loadAllConfigs ()	void	Membaca semua file konfigurasi sekaligus ke dalam atribut-atribut map
getPropertyData (std::string code) const	std::map<std::string, std::string>	Mengembalikan seluruh data properti berdasarkan kode (misal: 'JKT')
getRailroadRent (int count) const	int	Mengembalikan biaya sewa railroad berdasarkan jumlah yang dimiliki
getUtilityMultiplier (int count) const	int	Mengembalikan faktor pengali utility berdasarkan jumlah yang dimiliki
getTaxData (std::string type) const	int	Mengembalikan nilai pajak berdasarkan tipe ('PPH_FLAT', dll)
getSpecialData (std::string	int	Mengembalikan nilai petak spesial

<code>type) const</code>		berdasarkan tipe ('GO_SALARY', dll)
<code>getMaxTurn() const</code>	<code>int</code>	Getter untuk batas maksimum giliran
<code>getInitialBalance() const</code>	<code>int</code>	Getter untuk saldo awal pemain

2.4.2. Class SaveManager

Kelas SaveManager adalah kelas yang menangani serialisasi dan deserialisasi seluruh state permainan ke/dari `file.txt` sesuai format yang ditentukan di spesifikasi

Nama Atribut	Return Type	Deskripsi
<code>saveDirectory</code>	<code>std::string</code>	Direktori default penyimpanan file save

Nama Method	Return Type	Deskripsi
<code>SaveManager(std::string dir)</code>	constructor	Menerima path direktori penyimpanan
<code>saveGameState(const std::string& filename, const GameManager& gm)</code>	<code>bool</code>	Menyimpan seluruh state <code>GameManager</code> ke file; Mengembalikan true jika berhasil
<code>loadGameState(cost std::string& filename,</code>	<code>bool</code>	Memuat state dari file ke <code>GameManager</code> , mengembalikan

GameManager& gm)		true jika berhasil
------------------	--	--------------------

2.4.3. Class TransactionLogger

Kelas TransactionLogger adalah kelas yang mencatat setiap kejadian signifikan permainan secara otomatis di memori. Log disimpan ke file saat game di-save dan dapat ditampilkan dengan perintah CETAK_LOG.

Nama Atribut	Return Type	Deskripsi
logEntries	std::vector<std::string>	Daftar seluruh entri log yang tersimpan selama sesi berlangsung

Nama Method	Return Type	Deskripsi
logAction(int turn, std::string username, std::string action, std::string detail)	void	Menambahkan satu entri log baru dengan format [Turn N] username AKSI detail
printLogs(int count = -1) const	void	Menampilkan log; jika count = -1 tampilkan semua, jika count > 0 tampilkan N terakhir
loadGameState(const std::string& filename, GameManager& gm)	const std::vector<std::string>&	Mengembalikan referensi ke seluruh log untuk keperluan serialisasi save

<code>clearLogs()</code>	<code>void</code>	Mengosongkan seluruh log (digunakan saat memuat game baru)
--------------------------	-------------------	---

2.5. Kelas Tile Hierarchy

2.5.1. Class Tile

Kelas Tile adalah kelas abstrak dasar untuk semua petak di papan. Mendefinisikan interface `onLanded()` yang wajib di implementasi setiap jenis petak sesuai mekanismenya masing-masing.

Nama Atribut	Return Type	Deskripsi
<code>index</code>	<code>int</code>	Indeks posisi petak di papan (1-40)
<code>code</code>	<code>std::string</code>	Kode unik petak (misal: 'JKT', 'GO', 'PPH')
<code>name</code>	<code>std::string</code>	Nama lengkap petak(misal : 'Jakarta', 'Petak Mulai')
<code>type</code>	<code>std::string</code>	Tipe petak('STREET', 'RAILROAD', 'UTILITY', 'TAX', dll)

Nama Method	Return Type	Deskripsi
<code>Tile(int idx, std::string c, std::string n, std::string t)</code>	constructor	Menginisialisasi atribut dasar petak
<code>~Tile()</code>	destructor	Membersihkan petak tile
<code>getIndex() const</code>	int	Getter indeks petak
<code>getCode() const</code>	std::string	Getter kode petak
<code>getName() const</code>	std::string	Getter nama petak
<code>getType() const</code>	std::string	Getter tipe petak
<code>onLanded(Player& player, GameManager& gm) = 0</code>	void(pure virtual)	Dipanggil saat pemain mendarat di petak ini; efek berbeda per subclass

2.5.2. Class PropertyTile : public Tile

Kelas PropertyTile adalah pewaris dari kelas Tile dan juga kelas abstrak untuk semua petak properti yang dapat dimiliki pemain (street, Railroad, utility). Mengelola status kepemilikan dan gadai, serta mendefinisikan interface calculateRent()

Nama Atribut	Return Type	Deskripsi
<code>buyPrice</code>	int	Harga beli properti
<code>mortgageValue</code>	int	Nilai uang yang diterima pemain saat menggadaikan properti

ownerName	std::string	Username pemilik properti; 'BANK' jika belum dimiliki
propertyStatus	std::string	Status properti: 'BANK', 'OWNED', atau 'MORTGAGED'

Nama Method	Return Type	Deskripsi
PropertyTile(int idx, std::string c, std::string n, std::string t, int price, int mortgage)	constructor	Menginisialisasi data dasar properti
calculateRent(Dice& dice, int ownerPropertyCount) = 0	int(pure virtual)	Menghitung sewa berdasarkan kondisi properti, implementasi tiap subclass
getBuyPrice() const	int	Getter harga beli
getMortgageValue() const	int	Getter nilai gadai
getOwner() const	std::string	Getter username properti
setOwner(std::string owner)	void	Mengubah pemilik properti
getPropertyStatus() const	std::string	Getter status properti
mortgageProperty()	void	Menggadaikan properti: ubah status ke MORTGAGED
redeemProperty()	void	Menebus properti: ubah status kembali ke OWNED

<code>onLanded(Player& player, GameManager& gm) override</code>	<code>void</code>	Menangani pendaratan di properti: tawarkan beli jika BANK, bayar sewa jika OWNED
---	-------------------	--

2.5.3. Class StreetTile : public PropertyTile

Kelas StreetTile adalah pewaris dari kelas PropertyTile yang menjadi petak lahan/street dengan color group. Mendukung pembangunan rumah dan hotel, efek festival, serta monopoli color group untuk sewa ganda.

Nama Atribut	Return Type	Deskripsi
<code>colorGroup</code>	<code>std::string</code>	Kelompok warna properti (misal: 'HIJAU', 'BIRU TUA')
<code>rentTable</code>	<code>std::vector<int></code>	Tabel sewa per level bangunan: [L0, L1_rumah, L2_rumah, ..., hotel]
<code>housePrice</code>	<code>int</code>	Biaya membangun 1 rumah (dan upgrade ke hotel)
<code>buildingCount</code>	<code>int</code>	Jumlah bangunan saat ini: 0-4 (rumah_ atau 5(hotel, kode 'H'))
<code>festivalMultiplier</code>	<code>int</code>	Pengali sewa akibat festival: 1, 2, 4, atau 8
<code>festivalDuration</code>	<code>int</code>	Sisa giliran efek festival aktif (0 = tidak aktif)

Nama Method	Return Type	Deskripsi
<code>StreetTile(int idx, std::string c, std::string n, int price, int mortgage, std::string color, int hPrice, std::vector<int> rents)</code>	constructor	Menginisialisasi semua atribut street dari data konfigurasi
<code>calculateRent(Dice& dice, int ownerPropertyCount) override</code>	int	Menghitung sewa berdasarkan level bangunan, monopoli, dan multiplier festival
<code>buildHouse()</code>	void	Menambah 1 bangunan; memvalidasi syarat pemerataan dan maksimal hotel
<code>demolishBuilding()</code>	void	Mengurangi 1 bangunan (saat likuidasi atau DemolitionCard)
<code>applyFestival(int multiplier, int duration)</code>	void	Mengaktifkan/memperkuat efek festival pada properti ini
<code>getColorGroup() const</code>	std::string	Mengurangi durasi festival sebesar 1 setiap awal giliran pemilik
<code>getBuildingCount() const</code>	int	Getter jumlah bangunan saat ini
<code>getTotalBuildingValue() const</code>	int	Menghitung total nilai bangunan (untuk likuidasi = harga beli bangunan)

2.5.4. Class RailroadTile : public PropertyTile

Kelas RailroadTile adalah pewaris dari kelas PropertyTile yang menjadi petak stasiun kereta. Kepemilikan otomatis tanpa biaya bagi pemain pertama yang mendarat. Sewa bergantung pada jumlah railroad yang dimiliki pemiliknya

Nama Atribut	Return Type	Deskripsi
rentTable	std::Map<int, int>	Tabel sewa berdasarkan jumlah railroad yang dimiliki (1→M25, 2→M50, dll)

Nama Method	Return Type	Deskripsi
UtilityTile(int idx, std::string c, std::string n, int price, int mortgage, std::map<int, int> rents)	constructor	Menginisialisasi dengan tabel sewa dari konfigurasi
calculateRent(Dice& dice, int ownerPropertyCount) override	int	Mencari sewa di rentTable berdasarkan ownerCount
onLanded(Player& player, GameManager& gm) override	void	Override: jika BANK, langsung diberikan ke pemain tanpa biaya; jika OWNED, bayar sewa

2.5.5. Class UtilityTile : public PropertyTile

Kelas `UtilityTile` adalah pewaris dari Kelas `PropertyTile` yang menjadi petak utilitas (PLN dan PAM). Kepemilikan otomatis tanpa biaya. Sewa = total dadu x faktor pengali sesuai jumlah utility yang dimiliki pemilik.

Nama Atribut	Return Type	Deskripsi
<code>rentTable</code>	<code>std::Map<int, int></code>	Tabel faktor pengali sewa berdasarkan jumlah utility yang dimiliki (1→4x, 2→10x)

Nama Method	Return Type	Deskripsi
<code>UtilityTile(int idx, std::string c, std::string n, int price, int mortgage, std::map<int, int> multipliers)</code>	constructor	Menginisialisasi dengan tabel pengali dari konfigurasi
<code>calculateRent(Dice& dice, int ownerPropertyCount) override</code>	int	Menghitung sewa = <code>dice.getTotal() x multiplierTable[ownerCount]</code>
<code>onLanded(Player& player, GameManager& gm) override</code>	void	Override: jika BANK, langsung diberikan ke pemain; jika OWNED, hitung dan bayar sewa

2.5.6. Class TaxTile : public Tile

Kelas `TaxTile` adalah pewaris dari Kelas `Tile` yang menjadi petak pajak (PPH dan PBM). Tidak dapat dimiliki pemain. PPH menawarkan pilihan flat atau persentase kekayaan; PBM langsung dipotong flat

Nama Atribut	Return Type	Deskripsi
<code>taxType</code>	<code>std::string</code>	Tipe pajak: 'PPH' atau 'PBM'
<code>flatTax</code>	<code>int</code>	Nilai pajak flat dari konfigurasi
<code>percentageTax</code>	<code>int</code>	Persentase pajak terhadap total kekayaan (hanya untuk PPH)

Nama Method	Return Type	Deskripsi
<code>TaxTile(int idx, std::string c, std::string n, std::string type, int flat, int pct)</code>	constructor	Menginisialisasi tipe dan nilai pajak dari konfigurasi
<code>onLanded(Player& player, GameManager& gm) override</code>	<code>void</code>	PPH : tampilkan pilihan dan proses pembayaran; PBM : langsung potong flat

2.5.7. Class CardTile : public Tile

Kelas `CardTile` adalah pewaris dari Kelas `Tile` yang menjadi petak kartu kesempatan atau Dana Umum. Saat pemain mendarat, otomatis mengambil kartu teratas dari deck yang sesuai dan mengeksekusinya.

Nama Atribut	Return Type	Deskripsi
<code>deckType</code>	<code>std::string</code>	Tipe deck: 'CHANCE' atau 'COMMUNITY_CHEST'

Nama Method	Return Type	Deskripsi
<code>CardTile(int idx, std::string c, std::string n, std::string type)</code>	constructor	Menginisialisasi dengan tipe deck
<code>onLanded(Player& player, GameManager& gm) override</code>	<code>void</code>	Mengambil kartu dari deck yang sesuai lalu memanggil <code>card->execute()</code>

2.5.8. Class FestivalTile : public Tile

Kelas `FestivalTile` adalah pewaris Kelas `Tile` yang menjadi petak festival. Saat pemain mendarat, pemain memiliki properti yang dimilikinya untuk mendapatkan penggandaan sewa selama 3 giliran.

Nama Method	Return Type	Deskripsi
<code>FestivalTile(int idx, std::string c, std::string n)</code>	constructor	Menginisialisasi petak festival
<code>onLanded(Player& player, GameManager& gm) override</code>	void	Menampilkan daftar properti pemain, meminta pilihan, lalu memanggil <code>applyFestival()</code>

2.5.9. Class GoTile : public Tile

Kelas `GoTile` adalah pewaris dari kelas `Tile` yang menjadi petak Mulai (GO). Memberikan gaji kepada pemain yang berhenti tepat disini atau melewatinya. Titik awal permainan semua pemain.

Nama Atribut	Return Type	Deskripsi
<code>salaryAmount</code>	int	Besaran gaji yang diterima pemain dari konfigurasi (GO_SALARY)

Nama Method	Return Type	Deskripsi
<code>GoTile(int idx, std::string c, std::string n, int salary)</code>	constructor	Menginisialisasi petak GO dengan nilai gaji
<code>onLanded(Player& player,</code>	void	Memberikan gaji saat pemain

<code>GameManager& gm) override</code>		berhenti tepat di petak GO
<code>passGo(Player& player, GameManager& gm)</code>	<code>void</code>	Memberikan gaji saat pemain melewati petak GO dalam perjalanan

2.5.10. Class JailTile : public Tile

Kelas `JailTile` adalah pewaris dari kelas `Tile` yang menjadi petak Penjara dengan dua mode: Hanya Mampir (tidak ada penalti) dan di Dalam penjara (status `JAILED`, harus keluar dengan bayar denda/kartu/double dadu).

Nama Method	Return Type	Deskripsi
<code>JailTile(int idx, std::string c, std::string n, int bail)</code>	<code>constructor</code>	Besaran denda untuk keluar dari penjara dari konfigurasi (<code>JAIL_FINE</code>)
<code>onLanded(Player& player, GameManager& gm) override</code>	<code>void</code>	Jika status <code>ACTIVE</code> : hanya mampir. Jika <code>JAILED</code> ; jalankan <code>processInmate()</code>
<code>processInmate(Player& player, GameManager& gm)</code>	<code>void</code>	Menangani mekanisme keluar penjara : bayar denda, kartu bebas, atau dadu double

2.5.11. Class FreeParkingTile : public Tile

Kelas `FreeParkingTile` adalah pewaris dari kelas `Tile` yang menjadi petak Bebas parkir. Tidak ada aksi apapun saat pemain mendarat di sini - berfungsi sebagai rest area.

Nama Method	Return Type	Deskripsi
<code>FreeParkingTile(int idx, std::string c, std::string n)</code>	constructor	Menginisialisasi petak bebas parkir
<code>onLanded(Player& player, GameManager& gm) override</code>	void	Tidak melakukan apa-apa (<i>rest area</i>)

2.5.12. Class GoToJailTile : public Tile

Kelas `GoToJailTile` adalah pewaris dari kelas `Tile` yang menjadi petak pergi ke Penjara. Saat pemain mendarat, bidak langsung dipindahkan ke `JailTile` dan status berubah menjadi `JAILED`. Giliran langsung berakhir

Nama Method	Return Type	Deskripsi
<code>GoToJailTile(int idx, std::string c, std::string n)</code>	constructor	Menginisialisasi petak pergi ke penjara
<code>onLanded(Player& player, GameManager& gm) override</code>	void	Memindahkan pemain ke petak penjara dan mengubah status menjadi <code>JAILED</code>

2.6. Kelas GameState Hierarchy

2.6.1. Class GameState

Kelas GameState adalah kelas abstrak dasar yang mendefinisikan antarmuka untuk mesin status (state machine) permainan. Setiap status (seperti lelang, giliran berjalan, atau pemilihan skill) mewarisi kelas ini untuk menangani logika input dan pembaruan UI secara spesifik.

Nama Method	Return Type	Deskripsi
<code>onEnter(gm)</code>	<code>void</code>	Virtual method yang dipanggil sekali saat state baru dimasukkan; digunakan untuk inisialisasi data.
<code>handleInput(gm, gui)</code>	<code>void (pure virtual)</code>	Menangani masukan dari pengguna (keyboard/mouse) yang relevan dengan status saat ini.
<code>updateUI(gm, gui)</code>	<code>void (pure virtual)</code>	Bertanggung jawab untuk merender elemen visual spesifik yang berkaitan dengan status tersebut.

2.6.2. Class StateAuction : public GameState

Kelas StateAuction menangani alur mekanisme lelang properti ketika pemain mendarat di petak yang tidak dibeli atau saat proses likuidasi aset.

Nama Atribut	Return Type	Deskripsi
<code>property</code>	<code>PropertyTile*</code>	Pointer ke properti yang sedang dilelang.
<code>participants</code>	<code>std::vector<Player*></code>	Daftar pemain yang masih ikut serta dalam lelang.
<code>currentBid</code>	<code>int</code>	Nilai tawaran tertinggi saat ini.
<code>highestBidder</code>	<code>Player*</code>	Pointer ke pemain yang memegang tawaran tertinggi.
<code>currentPlayerIndex</code>	<code>int</code>	Indeks pemain yang sedang mendapat giliran menawar.

Nama Method	Return Type	Deskripsi
<code>StateAuction(prop, players)</code>	<code>constructor</code>	Menginisialisasi data lelang, peserta, dan bid awal (M10).
<code>nextPlayer()</code>	<code>void</code>	Memindahkan giliran ke peserta lelang berikutnya.
<code>endAuction(gm)</code>	<code>void</code>	Menyelesaikan lelang, mentransfer properti ke pemenang, dan memotong saldo.
<code>handleInput(gm, gui)</code>	<code>void</code>	Menangani bid (+M10/+M50) atau opsi menyerah, serta logika otomatis untuk pemain COM.

<code>updateUI (gm, gui)</code>	<code>void</code>	Menampilkan panel visual ruang lelang dan informasi penawaran terkini.
---------------------------------	-------------------	--

2.6.3. Class StateLiquidation : public GameState

Kelas StateLiquidation muncul ketika pemain memiliki kewajiban pembayaran (sewa/pajak) yang melebihi saldo tunai mereka, memaksa pemain untuk menjual atau menggadaikan aset.

Nama Atribut	Return Type	Deskripsi
<code>requiredAmount</code>	<code>int</code>	Jumlah total uang yang harus dilunasi pemain.

Nama Method	Return Type	Deskripsi
<code>onEnter (gm)</code>	<code>void</code>	Mengambil jumlah utang dari status pemain saat ini.
<code>handleInput (gm, gui)</code>	<code>void</code>	Memproses aksi penggadaian properti atau pernyataan bangkrut jika aset habis.
<code>updateUI (gm, gui)</code>	<code>void</code>	Menampilkan daftar aset yang bisa dilikuidasi dan indikator pemenuhan utang.

2.6.4. Class StatePromptUpgrade: public GameState

Kelas StatePromptUpgrade menangani logika pembangunan rumah dan upgrade hotel pada properti jenis Street yang telah dimonopoli.

Nama Atribut	Return Type	Deskripsi
monopolyGroups	map<string, vector<StreetTile*>>	Data grup warna yang telah dikuasai penuh oleh pemain.
phase	Enum Phase	Tahapan pemilihan (Pilih Grup, Pilih Petak, Konfirmasi Hotel).

Nama Method	Return Type	Deskripsi
buildMonopolyGroups (gm)	void	Memindai papan untuk menentukan grup warna mana yang berhak dibangun bangunan.
canBuildOn (target)	void	Mengecek validasi syarat pemerataan bangunan sesuai aturan permainan.
handleInput (gm, gui)	void	Memproses seleksi properti dan transaksi pembangunan bangunan.
updateUI (gm, gui)	void	Menampilkan menu interaktif pembangunan dan status level bangunan saat ini.

2.6.5. Class StateJailed: public GameState

Kelas StateJailed mengatur pilihan pemain saat berada di dalam penjara pada awal gilirannya.

Nama Method	Return Type	Deskripsi
<code>handleInput(gm, gui)</code>	<code>void</code>	Memproses pilihan keluar: membayar denda M50 atau mengocok dadu untuk mencari double.
<code>updateUI(gm, gui)</code>	<code>void</code>	Menampilkan dialog status penjara dan opsi aksi yang tersedia.

2.6.6. Class StateWaitingRoll: public GameState

Status default pada awal giliran pemain di mana pemain dapat melakukan aksi manajerial atau memulai pergerakan.

Nama Method	Return Type	Deskripsi
<code>handleInput(gm, gui)</code>	<code>void</code>	Menangani kocok dadu (SPACE), menyimpan permainan (S), atau masuk ke mode perintah (C).
<code>transitionAfterMove(gm, p)</code>	<code>void</code>	Menentukan status berikutnya berdasarkan jenis petak tempat bidak mendarat.
<code>updateUI(gm, gui)</code>	<code>void</code>	Menampilkan petunjuk tombol aksi utama di layar.

2.6.7. Class StateTeleportSelect: public GameState

Status khusus yang dipicu oleh TeleportCard, memungkinkan pemain berpindah ke koordinat petak manapun.

Nama Method	Return Type	Deskripsi
<code>handleInput(gm, gui)</code>	<code>void</code>	Menangkap input teks berupa ID petak dan memindahkan bidak ke tujuan.
<code>updateUI(gm, gui)</code>	<code>void</code>	Menampilkan kotak input teks untuk koordinat teleportasi.

2.6.8. Class StateTurnEnded: public GameState

Status antara setelah pemain melakukan aksi petak sebelum giliran benar-benar beralih ke pemain berikutnya.

Nama Method	Return Type	Deskripsi
<code>handleInput(gm, gui)</code>	<code>void</code>	Menangani transisi ke giliran berikutnya atau giliran tambahan jika mendapat double.
<code>updateUI(gm, gui)</code>	<code>void</code>	Menampilkan rangkuman log aksi terakhir dan tombol konfirmasi selesai.

2.6.9. Class StateDemolishSelect: public GameState

Kelas StateDemolishSelect menangani logika penggunaan DemolitionCard. Status ini melakukan pemindaian terhadap seluruh papan untuk mencari bangunan milik lawan yang valid untuk dihancurkan.

Nama Atribut	Return Type	Deskripsi
targetProperties	vector<StreetTile*>	Daftar properti milik lawan yang memiliki setidaknya satu bangunan.
selectedProperty	StreetTile*	Pointer ke properti yang dipilih oleh pemain untuk dihancurkan.
phase	Enum Phase	Tahapan status: SELECT_TARGET (memilih) atau CONFIRM_DEMOLISH (konfirmasi).

Nama Method	Return Type	Deskripsi
onEnter (gm)	void	Melakukan inisialisasi awal dan memanggil scanTargets() untuk memfilter properti lawan.
scanTargets (gm)	void	Logika pencarian petak STREET yang dimiliki lawan dan memiliki bangunan (rumah/hotel).
handleInput (gm, gui)	void	Menangani navigasi daftar target dan eksekusi fungsi demolishBuilding() pada properti terpilih.

Nama Method	Return Type	Deskripsi
<code>updateUI(gm, gui)</code>	<code>void</code>	Menampilkan panel besar berisi daftar properti lawan beserta status jumlah bangunan yang bisa dihancurkan.

2.6.10. Class StateDropCard: public GameState

Kelas StateDropCard dipicu secara otomatis oleh sistem ketika seorang pemain mendapatkan kartu keempat, sementara batas maksimal kartu di tangan adalah tiga. Status ini memaksa pemain untuk memilih satu kartu untuk dibuang sebelum dapat melanjutkan permainan..

Nama Method	Return Type	Deskripsi
<code>handleInput(gm, gui)</code>	<code>void</code>	Mendeteksi penekanan tombol angka [1-4] untuk menentukan kartu mana yang akan dihapus dari vector tangan pemain.
<code>updateUI(gm, gui)</code>	<code>void</code>	Menampilkan panel peringatan berwarna oranye dan daftar deskripsi kartu yang saat ini dimiliki oleh pemain.

2.6.11. Class StateLassoSelect: public GameState

Kelas StateLassoSelect adalah status aktif ketika pemain menggunakan LassoCard. Status ini memungkinkan pemain untuk menarik posisi lawan ke petak yang sama dengan pengguna kartu, yang berpotensi memicu pembayaran sewa atau efek petak lainnya bagi korban.

Nama Method	Return Type	Deskripsi
<code>handleInput(gm, gui)</code>	<code>void</code>	Memungkinkan pemilihan pemain lawan melalui input angka. Setelah terpilih, posisi korban diubah dan fungsi <code>onLanded()</code> pada petak tujuan dipicu secara paksa.
<code>updateUI(gm, gui)</code>	<code>void</code>	Menampilkan antarmuka pemilihan target "Mode Lasso" di sisi kanan layar (Panel Info).

2.7. Kelas UI Layer**2.7.1. Class GameGUI**

Kelas `GameGUI` adalah lapisan antarmuka pengguna berbasis raylib. Murni menangani input/output dari terminal. Tidak mengandung logika permainan – seluruh aksi didelegasikan ke `GameManger`.

Nama Atribut	Data Type	Deskripsi
gm	GameManager&	Referensi ke GameManager untuk mengakses dan memanipulasi state permainan
screenWidth	const int	Lebar GUI
screenHeight	const int	Tinggi GUI
tileSize	const int	Ukuran tile dalam board
boardOffset	const int	Offset board dari ujung GUI
activeDeed	PropertyTile*	Referensi ke property yang aktif
showDeedPopup	bool	Apakah deed properti ditampilkan
showSkillMenu	bool	Apakah menu kartu yang ada ditampilkan
showAssetList	bool	Apakah daftar aset ditampilkan
isCommandMode	bool	Apakah mode command aktif
commandText	std::string	
TilePos	struct {int x, y;}	Posisi x dan y tile
tileMap	std::map<int, TilePos>	Struktur data yang memetakan indeks ke posisi tile

Nama Method	Return Type	Deskripsi
GameGUI (GameManager& manager)	constructor	Menerima referensi GameManager
initTileMap()		
drawTile(int index, Tile& tile)		
drawPlayers()		
drawLogBox()		
drawCenterLog()		
drawPlayerHUD()		
drawAssetList()		
render()	void	Menampilkan menu awal (New Game/Load Game) dan menginisialisasi permainan
displayBoard()	void	Menampilkan visualisasi papan permainan lengkap dengan warna, kepemilikan, dan posisi bidak
displayPropertyDeed(std::string code)	void	Menampilkan Akta Kepemilikan_properti berdasarkan kode petak (CETAK_AKTA)

<code>displayPlayerProperties(const Player& player)</code>	<code>void</code>	Menampilkan daftar properti yang dimiliki pemain aktif (CETAP_PROPRTI)
<code>promptLiquidation(Player& player, int deficit)</code>	<code>void</code>	Menampilkan panel likuidasi aset saat pemain tidak mampu membayar kewajiban

3. Justifikasi

Alasan Pemilihan Design

- Kami memilih design ini salah satunya adalah untuk pencegahan memory leak, memberikan inheritance dan virtual yang baik ke dalam proyek ini dengan memecah sebanyak banyaknya menjadi kelas anakan, serta pemakaian class generic yang pas (untuk card) sehingga dapat dipakai di semua card sesuai dengan spek yang ada. Desain ini juga mengadopsi 3-Layer Architecture (UI, Core, Data Access) untuk memastikan setiap komponen program terisolasi dan mudah untuk dikembangkan maupun diuji secara independen.

Kelebihan

- Abstraksi dan polimorfisme nya kuat karena kelas pada class pada `class Card` dan `class Tile` didefinisikan dengan `pure virtual (=0)` dan menjamin metodenya langsung mengarah ke kelas turunannya (memiliki implementasi unik).
- Pembagian kelas ke dalam **GameCLI** (UI), **GameManager** (Logic), dan **ConfigReader/SaveManager** (Data Access) sudah sesuai dengan instruksi spesifikasi untuk memisahkan tanggung jawab.

- Penggunaan atribut `private/protected` dengan `getter/setter` serta penerapan operator `+=` dan `>` pada kelas `Player` membuat manipulasi logika kekayaan pemain menjadi lebih bersih dan sesuai kaidah OOP
- Generic Class pada `template <typename T> class CardDeck` dapat terimplementasi dengan baik dan akan krusial pada Game Logiknya

Kekurangan

- Terdapat raw pointer pada kelas yang kami buat terutama kami menggunakan `std::vector<T*> deck` dan `std::vector<Tile*> tiles`.
- Circular Dependency (Ketergantungan Melingkar): Terdapat dependensi melingkar antara `Tile/Card` dengan `GameManager` melalui parameter fungsi `execute(..., GameManager& gm)` dan `onLanded(..., GameManager& gm)`.
- Efisiensi Passing Parameter: Pada `GameManager(ConfigReader cfg)`, objek `ConfigReader` dikirim secara *pass-by-value*. Karena kelas tersebut menyimpan banyak `std::map` yang berisi data konfigurasi properti, proses ini akan memicu penyalinan data yang berat.
- Terdapat Potensi "God Class" pada `GameGUI`: Karena semua perintah (21 perintah otomatis/manual) bermuara di `GameGUI`, kelas ini menjadi terlalu besar dan kompleks.

Alasan Perubahan Desain :

Pada desain awal, sistem sangat bergantung pada penggunaan `dynamic_cast` untuk mengecek tipe petak (misalnya mengecek apakah sebuah `Tile` adalah `PropertyTile` untuk dieksekusi fungsi belinya). Namun, pendekatan ini memiliki kekurangan besar:

1. **Overhead Performa:** `dynamic_cast` mengandalkan RTTI (*Run-Time Type Information*) yang membebani memori dan menurunkan FPS pada *game loop* utama, terutama saat GUI melakukan *rendering* terus-menerus.
2. **Pelanggaran OCP (Open-Closed Principle):** Setiap ada tipe petak baru, kode pemanggil harus terus dimodifikasi dengan rantai `if-else dynamic_cast`.

Perubahan yang Dilakukan: Kami mengubah arsitektur menjadi Pure Polymorphism menggunakan pendekatan *Fat Interface*. Kami menambahkan *virtual functions* dengan implementasi *default* di kelas dasar (`Tile` dan `PropertyTile`), seperti `canBuild()`, `getHousePrice()`, dan `applyFestival()`. Selain itu, kami memisahkan logika antarmuka dan *gameplay* secara drastis menggunakan State Pattern (`StatePromptBuy`, `StateAuction`, `StatePromptUpgrade`), sehingga `GameManager` tidak perlu mempedulikan render UI, melainkan hanya berpindah *State* saat *event* pendaratan (`onLanded`) memicunya.

4. Penerapan Konsep OOP**4.1. Inheritance & Polymorphism**

```
void GameManager::executeTurn(Player& currentPlayer) {
    if (currentPlayer.getStatus() == "BANKRUPT") return;
```

```
currentPlayer.takeTurn(*this);
}
```

Alasan Penggunaan: Polimorfisme memungkinkan *GameManager* untuk menyimpan semua petak di dalam satu `std::vector<Tile*>` dan memanggil fungsi `onLanded()` secara seragam. Setiap kelas turunan (seperti `StreetTile`, `TaxTile`, `GoTile`) memiliki implementasi `override` yang unik. Keuntungannya, jika di masa depan ada penambahan "Petak Baru", kode di `GameManager` tidak perlu diubah sama sekali (Mendukung *Open-Closed Principle*).

4.2. Method/Operator Overloading

```
bool Player::operator!() const {
    return status == "BANKRUPT";

// OPERATOR OVERLOADING
Player::operator bool() const {
    return status != "BANKRUPT";
}

Player& Player::operator++() {
    ++position;
    return *this;
}
```

```

Player& Player::operator--() {
    --position;
    return *this;
}

//CONTOH PENGGUNAAN

player -= rent;
*owner += rent;

```

Alasan Penggunaan: Penggunaan *operator overloading* (`+=` dan `-=`) membuat manipulasi saldo uang antarpemain menjadi sangat intuitif dan bersih secara sintaks. Hal ini merepresentasikan transaksi di dunia nyata ("Pemain A uangnya berkurang, Pemain B uangnya bertambah") tanpa perlu menggunakan fungsi panjang seperti `player.decreaseBalance(rent)`.

4.3. Template & Generic Classes

```

template <typename T>
class CardDeck {
private:
    std::vector<T*> deck;
    std::vector<T*> discardPile;

```

```

public:
    CardDeck() {}
    ~CardDeck() {
        for (T* card : deck) delete card;
        for (T* card : discardPile) delete card;
    }
    // dan metode lainnya
}
// contoh penggunaan di atribut gameManager:
CardDeck<ChanceCard> chanceDeck;
CardDeck<CommunityChestCard> communityDeck;
CardDeck<SkillCard> skillDeck;

```

Alasan Penggunaan: Mencegah duplikasi kode (*DRY - Don't Repeat Yourself*). Karena mekanisme pengocokan (*shuffle*), pengambilan (*draw*), dan pembuangan (*discard*) untuk semua jenis kartu (Chance, Community Chest, Skill) adalah sama, penggunaan *Template Class* mengizinkan satu cetak biru logika digunakan untuk berbagai tipe objek kartu yang berbeda dengan tipe data yang tetap aman (*Type-Safe*).

4.4. Exception

```

// CONTOH EXCEPTION TURUNAN EXCEPTION NIMONSPOLI

```

```

class InvalidPropertyException : public NimonspoliException {
private:
    std::string propertyCode;
public:
    InvalidPropertyException(std::string code)
        : NimonspoliException("Kode properti tidak ditemukan: " + code),
          propertyCode(code) {}

    std::string getCode() const { return propertyCode; }
};

// CONTOH PENGGUNAAN

map<string, string> ConfigReader::getPropertyData(string code) const {
    if (propertyConfig.find(code) != propertyConfig.end()) {
        return propertyConfig.at(code);
    }
    throw InvalidPropertyException(code);
}

```

Alasan Penggunaan: Kondisi di mana pemain tidak bisa membayar tagihan adalah kondisi anomali yang mengubah alur permainan secara drastis (masuk ke fase likuidasi aset atau kebangkrutan). Menggunakan Exception memisahkan *error handling* dari alur *gameplay* normal, sehingga fungsi `onLanded` tidak perlu penuh dengan *return value* bersarang.

4.5. C++ Standard Template Library

```
// contoh stl vector:
std::vector<Player*> activeBidders;
std::vector<T*> deck;
std::vector<T*> discardPile;
std::vector<std::string> logEntries;
std::vector<int> rentTable;

// contoh std map
std::map<int, int> multiplierTable;
std::map<int, int> rentTable;
```

STL digunakan secara masif dalam bentuk `std::vector` untuk penyimpanan data yang urutannya krusial (seperti urutan petak di papan dan daftar pemain), serta `std::map` untuk pencarian data konfigurasi dengan kompleksitas $O(\log n)$ menggunakan *Key-Value pair* (misal mencari data properti berdasarkan string kode petak seperti "JKT").

4.6. Konsep OOP lain

KONSEP LAIN YANG DIGUNAKAN : State Pattern & Abstract Base Class

```
// ABSTRACT CLASS UNTUK PERPINDAHAN STATE

class GameState {
public:
    virtual ~GameState() = default;
    virtual void onEnter(GameManager&) {}
    virtual void handleInput(GameManager& gm, GameGUI& gui) = 0;
    virtual void updateUI(GameManager& gm, GameGUI& gui) = 0;
};

// Implementasi perpindahan State

gm.changeState(std::make_unique<StatePromptUpgrade>());
```

Alasan Penggunaan: Kami menerapkan **State Design Pattern** untuk memisahkan logika antarmuka dan *flow* permainan. **GameState** bertindak sebagai *Abstract Base Class* murni. Keuntungannya, siklus permainan tidak lagi bergantung pada *looping* statis berantakan, melainkan berjalan mulus secara dinamis dari satu *State* ke *State* lain (misalnya dari **StateTurnEnded** ke **StateAuction** saat terjadi penolakan pembelian

5. Bonus Yang dikerjakan

5.1. Bonus yang diusulkan oleh spek

5.1.1. GUI

Kami mengganti CLI murni dengan integrasi *Graphical User Interface* (misal menggunakan Raylib). Papan permainan di-*render* secara 2D secara *real-time*, lengkap dengan *color-coding* kepemilikan properti, letak bidak pemain (warna/kode unik), dan tampilan HUD (Hitungan Dadu, Saldo, Status).

Pendekatan: Kami memisahkan layer presentasi di dalam kelas `GameGUI` dan menggunakan `updateUI()` di dalam *State Machine* agar panel-panel interaktif (seperti panel konfirmasi Beli, bayar Pajak, dan Lelang) muncul secara dinamis di tengah layar (*Pop-up*) tanpa memblokir *render* papan utama.

5.1.2. COM

Kami mengimplementasikan Bot AI yang mampu mengambil keputusan ekonomis secara otonom saat mendapatkan giliran.

Pendekatan: Melalui *override* `promptTurnAction()`, objek `ComputerPlayer` mengevaluasi *State* saat ini. Contoh penerapan logika cerdas COM:

- **Festival:** Bot secara otomatis mendeteksi properti miliknya dengan jumlah rumah terbanyak (`buildingCount` tertinggi) untuk dijadikan tempat festival, sehingga memaksimalkan keuntungan ganda.
- **Lelang (Auction):** Bot membatasi emosi. Ia mengkalkulasi harga wajar properti, dan hanya akan melakukan *bid* (+M10) jika harga penawaran saat ini masih di bawah batas toleransinya (Harga Dasar Properti + 30% dari Saldo Bot saat ini). Jika melampaui itu, Bot otomatis melakukan aksi *Fold*.

5.2. Bonus Kreasi Mandiri

Command Manual

- **Deskripsi Singkat:** Kami mengimplementasikan sistem pembaca perintah (Command Parser) manual di luar alur otomatis pelemparan dadu. Fitur ini memungkinkan pemain (atau *tester*) untuk memanipulasi *state* permainan secara langsung. Fitur ini sangat krusial untuk keperluan *debugging* dan pengujian skenario ujung (*edge-case testing*) tanpa harus bermain dari awal.
- **Pendekatan Implementasi:** Kami membangun metode pembaca input di dalam **GameCLI** yang dapat mem-parsing *string* dengan argumen. Beberapa perintah manual yang kami buat antara lain:
 - **ATUR_DADU <d1> <d2>**: Mem- *bypass* RNG (Random Number Generator) dadu dan memaksa dadu mengeluarkan angka spesifik.
 - **CETAK_LOG**: Memanggil metode `printLogs()` dari **TransactionLogger** untuk melihat riwayat transaksi ke belakang.
 - **CETAK_AKTA <kode>**: Menampilkan detail properti beserta level bangunan dan harga sewanya tanpa harus mendarat di petak tersebut.

6. Pembagian Tugas

Modul (dalam poin spek)	Implementer	Tester
Exception Handling	13524144	
Game Logic	13524130	13524130
Generic & Card hierarchy	13524147	13524147

Data Access Layer	13524147	
Tile Hierarchy	13524144	
GameState Hierarchy	13524130	
UI Layer	13524119, 13524130	13524130

Lampiran

Lampiran A. Form Asistensi :

<https://drive.google.com/file/d/1fwESVtRPIhSIx5J2CFDAVz-5tcyprjPe/view?usp=sharing>

Lampiran B. Diagram Kelas :

https://drive.google.com/file/d/1KuOc_p5CH2tbal8iTnkMQpDT4vkBYEO6/view?usp=sharing